MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

Formation Git et Gitlab

Apprendre à utiliser le gestionnaire de versions Git et la forge logicielle Gitlab en tant qu'utilisateur de R et RStudio

MINISTÈRE
DE L'AGRICULTURE
DE LA SOUVERAINETÉ
ALIMENTAIRE ET DE LA FORÊT
Liberté
Égalité
Fraternité

Sommaire

1 Pourquoi Git ?

2 Les bases de Git

3 Les zones de Git

4 Travailler avec une forge

5 Les branches

6 Pour en savoir plus

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

0.1 Avant-propos

Ce diaporama de formation a été rédigé dans le but d'être le support visuel des formations dispensées au MAASA.

Ces formations s'adressent à des agents **qui découvrent le contrôle de version**.
Elles sont données en présentiel sur une durée **d'une journée** à partir de juin 2024.

> ⓘ **Champ couvert par cette formation**
>
> **Ce support ne couvre pas tous les aspects du contrôle de version ni toutes les commandes Git.**
>
> De même, il est orienté pour être utile aux agents du SSM MAASA et se concentre donc sur une utilisation de R via RStudio qui est mise à disposition des agents sur la plateforme interne Cerise basée sur RStudio Workbench.
>
> L'utilisation avec d'autres IDE tels que VS Code ou encore Github Desktop n'est donc incluse.
>
> Enfin, en tant que forge interne du MAASA, les fonctionnalités principales de Gitlab sont présentées dans ce support. Même si la majorité des concepts de base sont équivalents, Github n'est pas abordé dans ce diaporama de formation.

1 Pourquoi Git ?

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

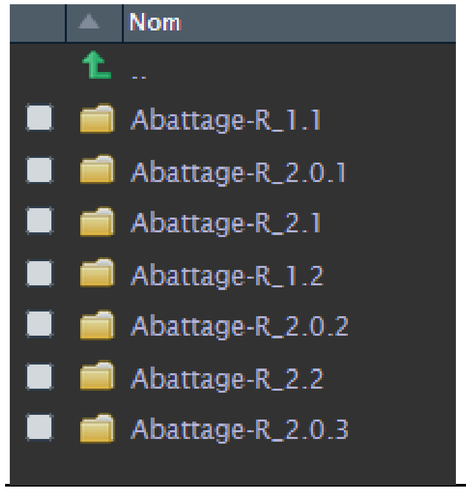
1.1 Le versioning

Principe général :

Le versioning consiste à conserver la version d'une entité logicielle quelconque de façon à pouvoir la retrouver facilement même après l'apparition et la mise en place de versions plus récentes.

Sans outil de versioning

Exemple de ce que l'on peut trouver sur un server :



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

1.2 Avantages du versioning

Le versioning permet de :

- Obtenir de **la traçabilité** : on sait qui a changé quoi, quand, comment et pourquoi
- **Travailler collectivement** sur les mêmes programmes **en même temps**
- **L'expérimentation** lors des développements sans risques (à l'aide des branches)
- Faire de la **relecture** (revues de codes…)
- **Revenir en arrière**

Avec une forge, il permet en plus de :

- D'archiver le code
- D'apporter de la visibilité à vos projets

=> Un usage qui se justifie même quand on est tout seul sur un projet !

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

1.3 Git

Parmi les logiciels de versioning existants, Git est le plus populaire depuis quelques années :

What are the primary version control systems you use?



Source : Stackoverflow

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

1.4 À propos de Git

Git est un logiciel libre de versioning.

Créé en 2005 par **Linus Torvalds** et utilisé pour le code source de Linux.

- Codé en C
- Plus de 1 400 000 commits en mars 2026 !



Source : Wikipédia

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2 Les bases de Git

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.1 Qu'est-ce qu'on versionne ?

On versionne les fichiers de type texte

Par exemple :

- Les programmes R, Python, SAS…
- La documentation au format texte, Markdown…
- Les fichiers quarto, Rmarkdown…
- Les fichiers de configuration de type yaml par exemple…

Éventuellement, on peut aussi versionner **des images** si on en a besoin dans une application ou une documentation.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.2 Qu'est-ce qu'on NE versionne PAS ? (1/2)

TOUT LE RESTE :)

C'est-à-dire notamment les fichiers tableurs, de traitements de texte, les pdf, les diaporamas de type powerpoint ou impress, les fichiers spécifiques aux projets R…

Pour se faciliter la tâche, on utilise un fichier spécifique `.gitignore` situé le plus souvent à la racine des projets.

Il s'agit d'un fichier texte que vous devez éditer, qui liste les fichiers et dossiers (sous forme d'expressions régulières) à ne pas versionner.

– une ligne = une règle ;

– on peut ignorer :
- des fichiers (exemple : `donnees.rds`)
- des dossiers (exemple : `data/`)
- des extensions (exemple : `*.xls`)
- …

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.3 Qu'est-ce qu'on NE versionne PAS ? (2/2)

Voici un exemple de fichier `.gitignore` qui peut servir de base pour une ré-utilisation :

```
.Rproj.user
.Rhistory
.RData
.Renviron
.Ruserdata

/* Les fichiers avec ces extensions
*.xls
*.xlsx
*.ods
*.pdf
*.docx
*.odt
*.ppt
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.4 Comment utiliser Git ?

2 façons seront présentées dans ce support :

- Via l'interface visuelle de l'IDE RStudio
  Pour les commandes les plus courantes du quotidien
- Via les commandes du terminal
  Pour les commandes plus avancées

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.5 Installer Git

- **Si vous travaillez avec Cerise**, Git est déjà installé.
  => Vous pouvez directement commencer à l'utiliser. (il vous faudra simplement régler votre configuration pour faire en sorte de dialoguer avec la forge interne Gitlab => voir **ici**)
- **Si vous travaillez en local**, consultez **cette procédure interne au MAASA**

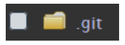
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.6 Git avec RStudio

Pré-requis : pour pouvoir utiliser correctement Git avec l'IDE RStudio, il convient d'utiliser le mode projet.

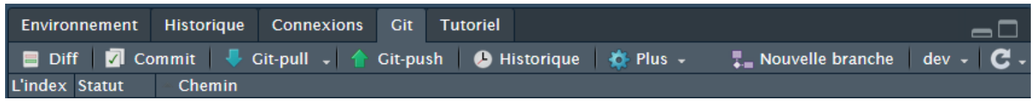
Comment savoir si un projet R est versionné avec Git ?

- Un fichier `.git` 

est présent dans l'explorateur de fichier.

> **Note**
>
> Pensez à cocher la case 

- À l'ouverture du projet R, un onglet Git s'affiche dans RStudio :



=> Une utilisation de Git en cliquant sur les boutons de RStudio

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

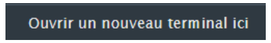
2.7 Git avec le terminal

- En local, vous pouvez directement utiliser les commandes du terminal sans RStudio.

Pour cela, il vous suffit de vous placer dans le bon répertoire (par exemple avec la commande `cd` pour `change directory`).

- Sur un server Posit comme Cerise, le terminal est désormais intégré dans les sessions RStudio à côté de la console classique de R 

Vous pouvez alors soit utiliser la même méthode qu'en local avec la commande `cd` soit de préférence :

- Vous placez dans le répertoire voulu dans l'explorateur de fichiers
- Cliquez sur le bouton `Plus` en haut à droite
- Cliquez sur le bouton 

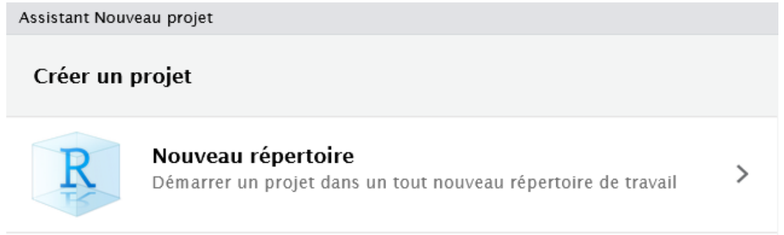
~~=> Une utilisation de Git en tapant des commandes qui commencent par git…~~

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

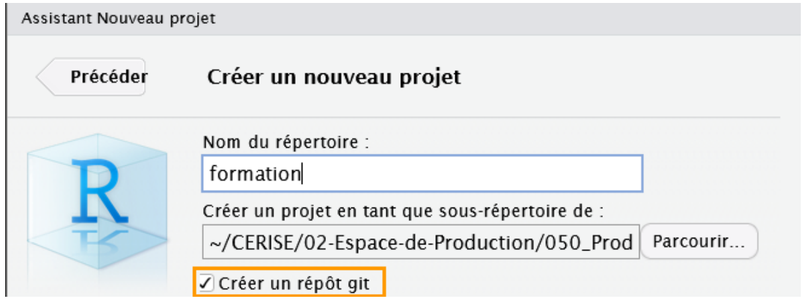
Liberté
Égalité
Fraternité

2.8 Initialiser un dossier local en dépôt Git avec RStudio

Créer un nouveau projet puis choisir un nouveau répertoire :



Après avoir choisi le nom du nouveau projet, cocher la case “Créer un répôt git” :



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.9 Initialiser un dossier local en dépôt Git avec le terminal

La commande `git init` permet d'initialiser un dossier avec Git

Cette commande crée un nouveau sous-répertoire nommé `.git` qui contient tous les fichiers nécessaires au dépôt — un squelette de dépôt Git.

Pour l'instant, aucun fichier n'est encore versionné !

Important

Il s'agit d'être vigilant sur l'emplacement du terminal au moment du lancement de la commande `git init` au risque d'initialiser le mauvais répertoire !

Astuce

Lors de l'initialisation du dépôt Git, on peut directement choisir le nom de la branche avec la commande `git init --initial-branch=<nom-de-branche>`.
=> voir partie 4 de ce support

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

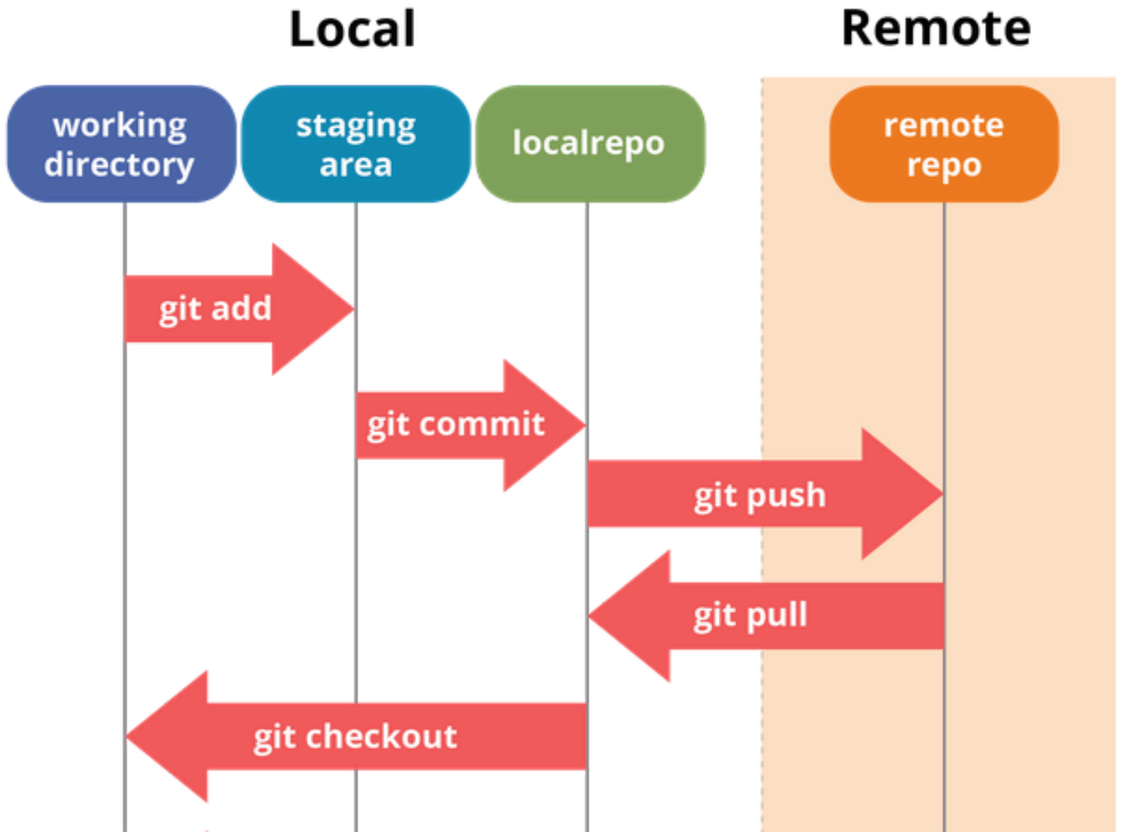
Liberté
Égalité
Fraternité

3 Les zones de Git

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.1 Fonctionnement



Source : itnext.io

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.2 Les zones de Git :

Pour évoquer le fonctionnement de Git et son articulation avec une forge, cela nécessite d'apprendre à utiliser de nouveaux termes (voir lexique diapo suivante).

- `working directory` : répertoire de travail de l'agent
- `staging area` ou `index` : zone tampon dans laquelle l'agent regroupe les changements en vue du prochain commit.
- `local repository` : dépôt local présent dans votre espace de travail (.git)
- `remote repository` : dépôt distant sur une forge

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.3 Les commandes principales de Git

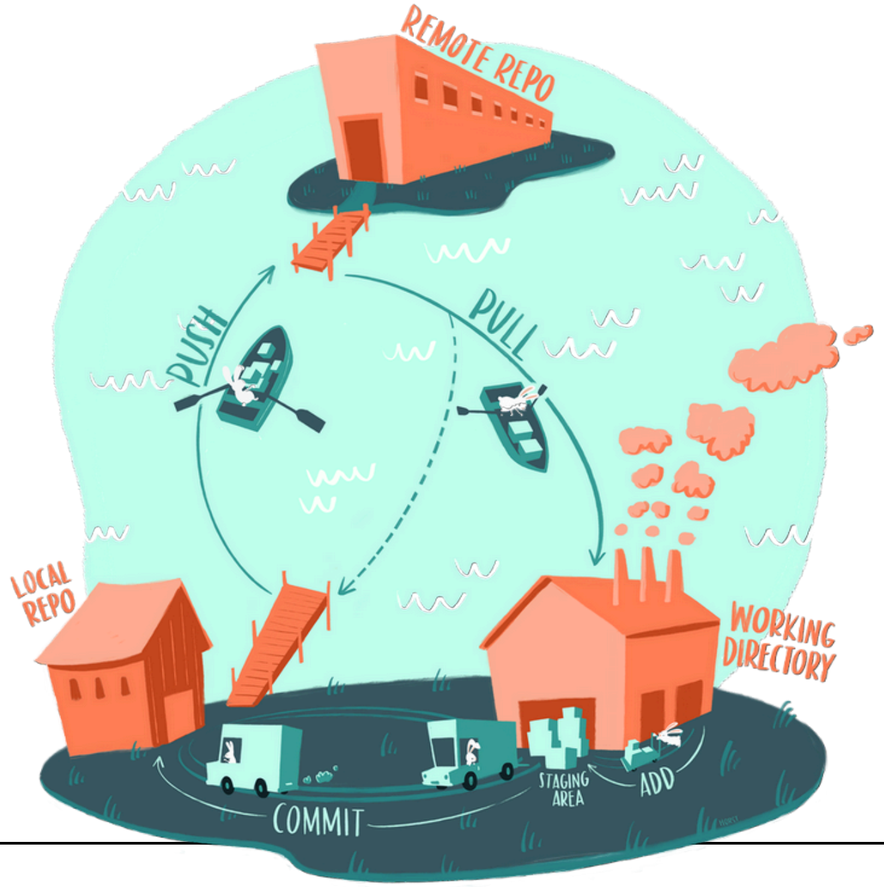
- `git add` : ajout des changements à l'index
- `git commit` : enregistre les changements placés dans l'index
- `git push` : envoie les changements vers le dépôt distant
- `git pull` : télécharge les changements depuis le dépôt distant
- `git checkout` : permet de se déplacer dans l'arbre (retour vers le passé, changement de branche)

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.4 Une 2ème illustration pour fixer les idées :)

Source : Allison Horst



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.5 Le working directory

Il s'agit de notre **copie de travail** c'est-à-dire le répertoire et sous-répertoires dans lesquels se trouvent les fichiers suivis par Git.

C'est là que l'agent effectue les changements sur les fichiers.
Il correspond à votre **espace de travail**.

> **Avis aux utilisateurs novices**
>
> Faire très attention au début de l'utilisation de Git : en fonction de la commande Git que vous allez utiliser (par exemple un `git checkout` pour changer de branche ou revenir dans le passé), **Git va changer le contenu de cet espace sans vous avertir**.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.6 La staging area ou l'index

Zone tampon dans laquelle l'agent regroupe les changements en vue du prochain commit.

L'agent peut **ajouter** ou **retirer** les changements à l'index.

Il peut faire cela en une seule ou plusieurs opérations (voir diapos suivantes).

> ⚠ **Attention**
>
> Ajouter des changements à plusieurs fichiers en même temps nécessite d'être sûr de ce que l'on fait et d'avoir correctement configuré le fichier `.gitignore` (voir diapositive suivante).

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

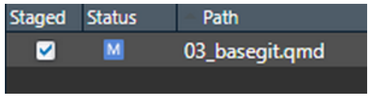
Liberté
Égalité
Fraternité

3.7 Ajouter des changements à l'index

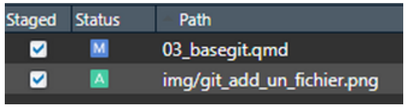
Pour ajouter des changements :

Avec RStudio

- Un seul fichier : **en cochant un fichier dans l'onglet Git**



- Plusieurs fichiers : **en cochant plusieurs fichiers dans l'onglet Git**



En ligne de commande :

- ~~Un seul fichier : `git add <file-name>`~~
- ~~Tous les fichiers : `git add .`~~

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.8 Retirer des changements à l'index

Pour retirer des changements :

Avec RStudio

En décochant les fichiers dans l'onglet Git

En ligne de commande :

- Un seul fichier : `git reset <file-name>`
- Tous les fichiers : `git reset`

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.9 Les états des fichiers avec RStudio

L'onglet de RStudio permet d'avoir des repères visuels sur les changements apportés à chacun de vos fichiers :



- `Deleted` : le fichier a été supprimé du working directory
- `Modified` : le contenu du fichier a été modifié
- `Untracked` : le fichier a été ajouté au working directory et Git ne l'a jamais vu auparavant

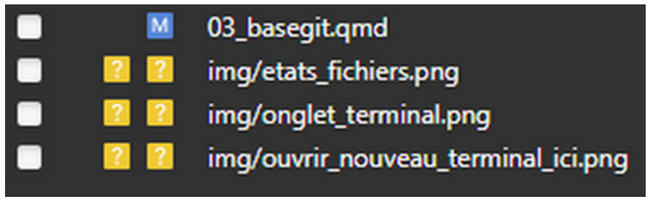
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.10 Les états des fichiers avec le terminal

La commande `git status` affiche l'état du working directory et de l'index.

Voici un exemple d'équivalence entre les icônes RStudio et le résultat renvoyé par la commande `git status` :




```
C:\Users\damien.dotta\DEMESIS\Formations_Git\formation_git_2024>git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   03_basegit.qmd

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        img/etats_fichiers.png
        img/onglet_terminal.png
        img/ouvrir_nouveau_terminal_ici.png

no changes added to commit (use "git add" and/or "git commit -a")
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

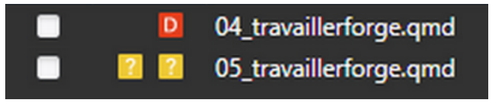
Liberté
Égalité
Fraternité

3.11 Les états des fichiers APRÈS l'ajout à l'index

Après l'ajout de fichier à l'index, 2 nouvelles icônes apparaissent dans RStudio (**même fonctionnement avec le terminal**) :



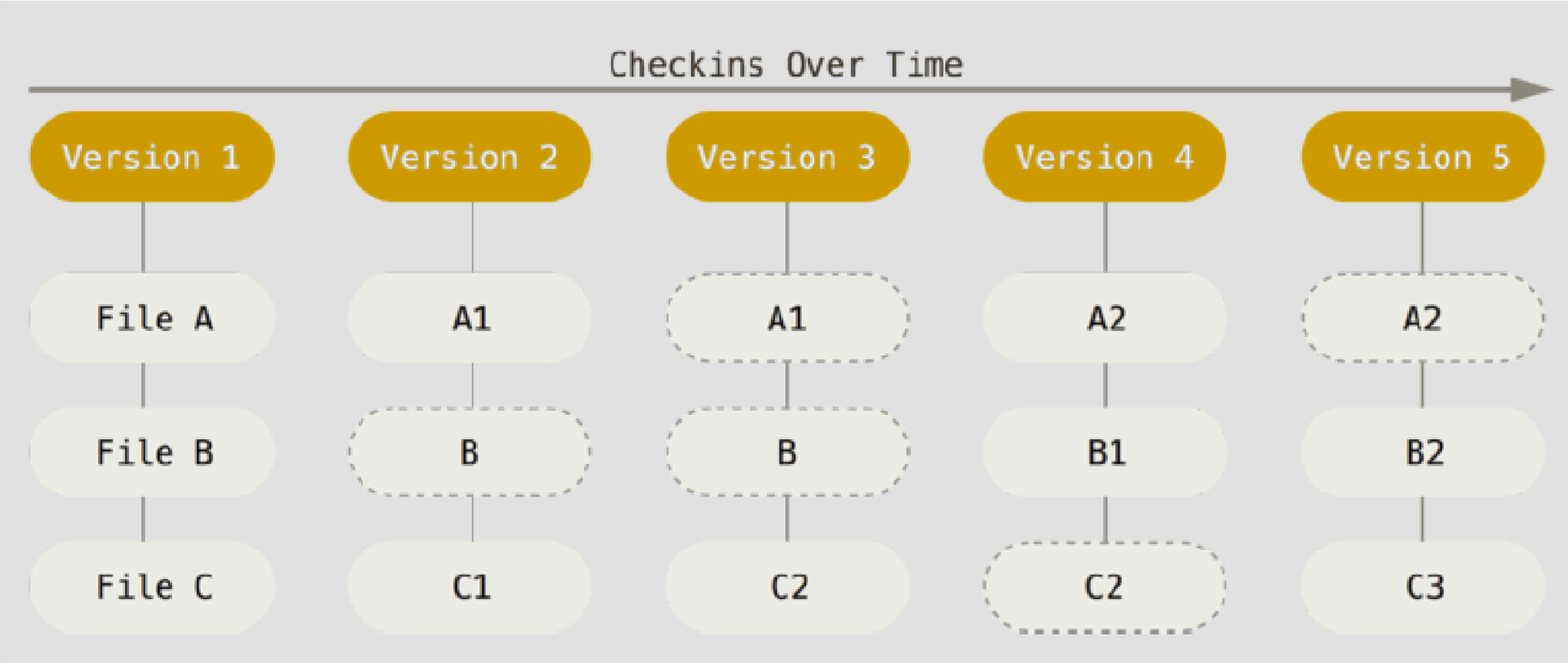
- `Added` : Git comprend que vous voulez ajouter le fichier au dépôt
- `Renamed` : Git comprend que le fichier a été renommé

Changement	AVANT l'ajout à l'index	APRÈS l'ajout à l'index
Ajout de fichier		
Renommage de fichier		

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.12 Comment Git stocke les différentes versions d'un fichier ?



À chaque fois que vous validez ou enregistrez l'état du projet dans Git, il prend un "snapshot" du contenu de votre working directory à ce moment et enregistre une référence à ce snapshot. Pour être efficace, si les fichiers n'ont pas changé, Git ne stocke pas le fichier à nouveau, juste une référence vers le fichier original qu'il a déjà enregistré.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

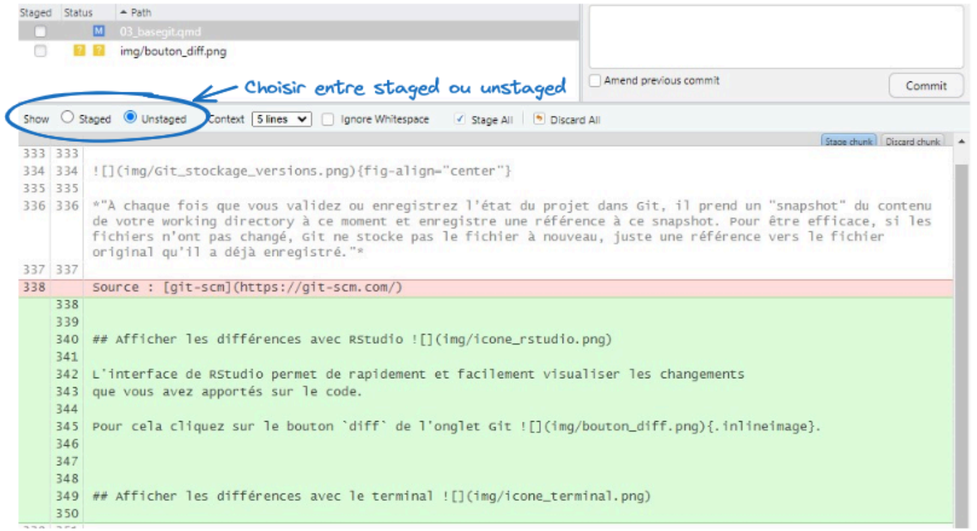
Liberté
Égalité
Fraternité

3.13 Afficher les différences avec RStudio

L'interface de RStudio permet de rapidement et facilement visualiser les changements que vous avez apportés sur le code.

Pour cela cliquez sur le bouton `diff` de l'onglet Git Diff.

- Le code supprimé s'affiche en rouge
- Le code ajouté s'affiche en vert



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.14 Afficher les différences avec le terminal

La commande `git diff` affiche les différences.

```
C:\Users\damien.dotta\DEMESIS\Formations_Git\formation_git_2024>git diff
diff --git a/03_basegit.qmd b/03_basegit.qmd
index 2977212..a76bb11 100644
--- a/03_basegit.qmd
+++ b/03_basegit.qmd
@@ -335,7 +335,19 @@ Après l'ajout de fichier à l'index, 2 nouvelles icônes apparaissent dans RStu

 *"À chaque fois que vous validez ou enregistrez l'état du projet dans Git, il prend un "snapshot" du contenu de votre working directory à ce moment et en
registre une référence à ce snapshot. Pour être efficace, si les fichiers n'ont pas changé, Git ne stocke pas le fichier à nouveau, juste une référence ve
rs le fichier original qu'il a déjà enregistré."*

-Source : [git-scm](https://git-scm.com/)
+
+
+## Afficher les différences avec RStudio ![](img/icone_rstudio.png)
+
+L'interface de RStudio permet de rapidement et facilement visualiser les changements
+que vous avez apportés sur le code.
+
+Pour cela cliquez sur le bouton `diff` de l'onglet Git ![](img/bouton_diff.png){.inlineimage}.
+
+
+
+## Afficher les différences avec le terminal ![](img/icone_terminal.png)
+
```

Pour afficher les différences sur les fichiers déjà ajoutés à l'index, la commande est `git diff --staged`.

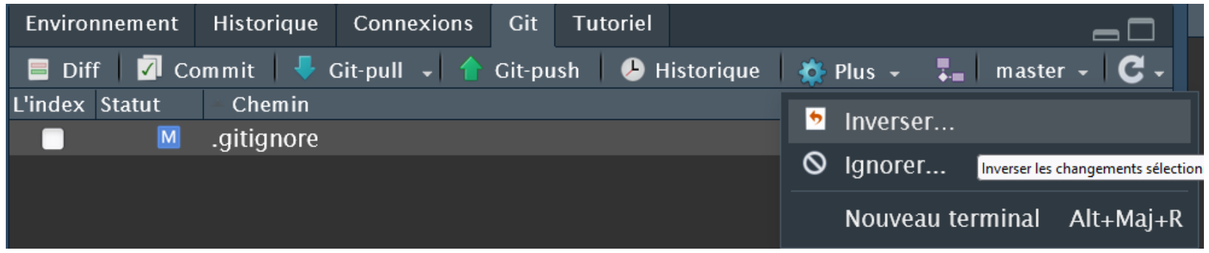
Le terminal permet d'aller plus loin dans l'observation des différences, par exemple en comparant 2 commits…

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

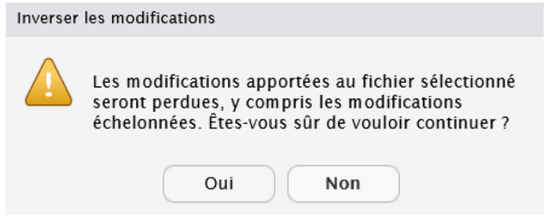
Liberté
Égalité
Fraternité

3.15 Inverser les changements avec RStudio

Dans RStudio, sélectionner le fichier sur lequel vous souhaitez inverser les changement puis cliquez sur le bouton `Plus` de l'onglet Git puis choisir le bouton `Inverser` :



Un message de confirmation vous demande si vous êtes sûr de votre action



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.16 Inverser les changements avec le terminal

La commande git restore permet de restaurer des fichiers

Action	Commande
Inverser les changements pas encore ajoutés à l'index	`git restore <file-name>`
Inverser les changements déjà ajoutés à l'index	`git restore --staged <file-name>`

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.17 Le commit

Action de figer l'état d'un dépôt

Un bon commit :

- Change une seule chose
- Peut être décrit avec un message clair et explicite

Erreur courante du débutant sur Git :

- Il code pendant des heures/jours/semaines
- Ajoute toutes les modifications à l'index avec `git add .`
- Puis il commite…

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.18 Caractéristiques du commit

- Un commit possède **un identifiant unique** (SHA)

Il informe les utilisateurs :

- Qui ? Quoi ? Où ? Quand ? => Avec ses **métadonnées**
- Quoi ? Où ? Comment ? => Avec l'affichage des **différences**
- Pourquoi ? => Avec **le message du commit**

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.19 Les messages de commit

D'après **Conventional Commits**, l'écriture des messages de commits devrait suivre la structure minimale suivante :

```
<type>: <description>
```

Le tableau suivant reprend les types les plus couramment utilisés :

Type Detail	Detail
wip	Développement d'une fonctionnalité
feat	Ajout d'une fonctionnalité
fix	Correction d'un bug/erreur
doc	Documentation
deprecated	Fonctionnalité dépréciée
chore	Tâches de routine/automatisées

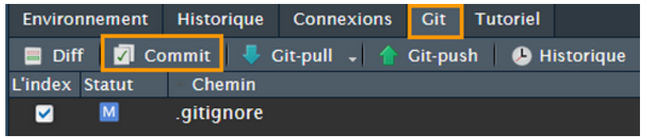
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

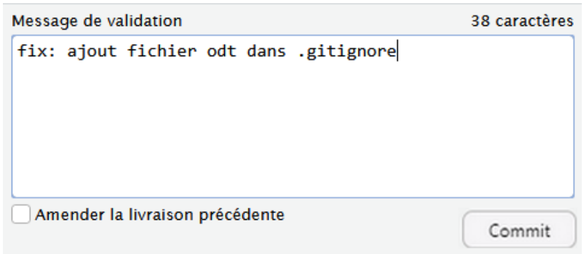
3.20 Comment committer avec RStudio ?

Après avoir ajouté les fichiers modifiés à l'index, il est temps de "committer" les changements c'est-à-dire de figer l'état de ces fichiers modifiés.

Dans RStudio, il faut cliquer sur le bouton "Commit" de l'onglet "Git"



Puis, renseigner dans la nouvelle fenêtre qui s'affiche le message du commit et cliquer sur le bouton "Commit" :



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.21 Comment committer avec le terminal ?

La commande `git commit -m "message du commit"` permet à la fois de faire le commit tout en renseignant le message associé

Voici un exemple :

```
git commit -m "fix: ajout fichier odt dans .gitignore"
```

> **Note**
>
> La commande `git commit` propose de nombreuses options qui sont disponibles ici.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.22 Exercice 1

Exercice 1

- Créer un projet R "formation-git" dans votre espace personnel Cerise en l'initialisant avec Git
- Vérifier que votre projet est bien versionné avec Git
- Ajouter les nouveaux fichiers à l'index et faire un 1er commit

- Créer des scripts R à l'intérieur de votre projet
- Enregistrer vos scripts
- Ajouter ces scripts à l'index et faire votre 2ème commit

BONUS : faire l'exercice avec RStudio et pour ceux qui sont en avance avec le terminal :)

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

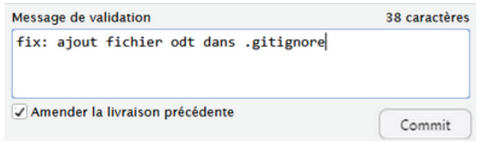
3.23 Modifier le dernier commit

Il y a au moins deux raisons de revenir sur un commit :

- Modifier le message de commit (même pour une faute de frappe)
- Ajouter ou supprimer des fichiers au commit.

R

Cocher la case `Amend previous commit` en dessous du message de commit précédent.



La commande à utiliser est :

```
git commit --amend -m 'Mon nouveau message de commit'
```

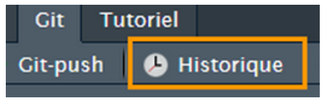
Attention ! Cette commande ne doit être effectuée que si vous n'avez pas encore poussé le commit sur dépôt distant sinon les autres utilisateurs du commit seront affectés puisque le SHA-1 est changé

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

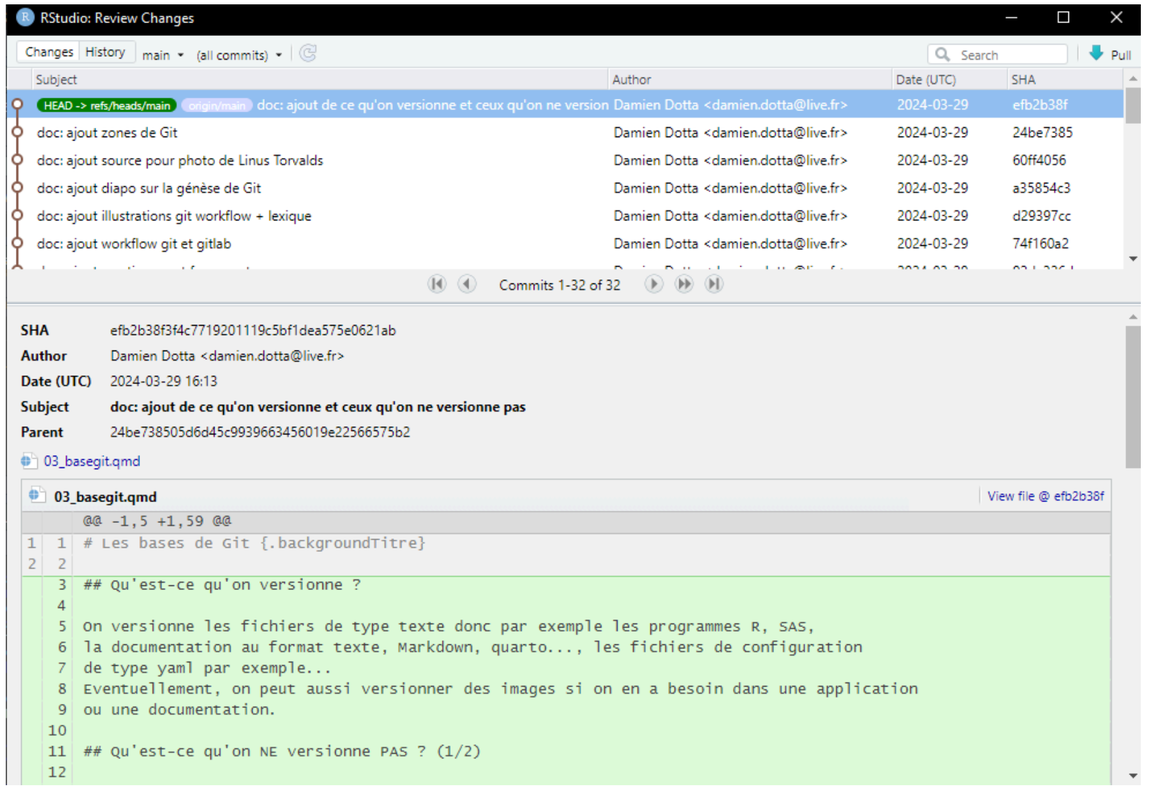
Liberté
Égalité
Fraternité

3.24 Consulter l'historique avec RStudio (1/2)

En cliquant sur le bouton `Historique` de RStudio :



Ce qui permet d'accéder à la fenêtre suivante :



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.25 Consulter l'historique avec RStudio (2/2)

RStudio permet facilement de :

- Balayer l'historique du projet en sélectionnant les différents commits.
- Accéder aux informations correspondants à chaque commit.
- Cliquer sur les fichiers concernés par chaque commit et afficher leur état au moment choisi.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.26 Consulter l'historique avec le terminal

Avec la commande `git log` : Exemple :

```
C:\Users\damien.dotta\DEMESIS\Formations_Git\formation_git_2024>git log
commit efb2b38f3f4c7719201119c5bf1dea575e0621ab (HEAD -> main, origin/main)
Author: Damien Dotta <damien.dotta@live.fr>
Date:   Fri Mar 29 17:13:00 2024 +0100

    doc: ajout de ce qu'on versionne et ceux qu'on ne versionne pas
```

Ce qu'on y trouve :

- l'identifiant du commit
- l'auteur du commit (nom + email)
- la date et l'heure du commit
- le message du commit

Note

La commande `git log --oneline` affiche les informations concernant l'historique de façon plus compacte mais moins riche.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.27 Fréquence des commits

Quelle fréquence pour les commit ?

- Aussi souvent que possible
- Regrouper les modifications en lots qui “font sens”, avec un message pertinent qui résume bien les modifications apportées
- **Il ne faut committer sur la base du temps passé mais plutôt sur la base des fonctionnalités.**

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.28 Exercice 2 (1/2)

Exercice 2 (1/2)

Reprendre l'exercice 1

- Apporter des changements dans un de vos scripts R
- Enregistrer votre script modifié
- Que se passe t'il dans l'onglet "Git" de RStudio ? Avec la commande `git status` ?
- Ajouter vos changements à l'index et faire un nouveau commit

- Supprimer un de vos scripts
- Que se passe t'il dans l'onglet "Git" de RStudio ? Avec la commande `git status` ?
- Ajouter vos changements à l'index et faire un nouveau commit

VOIR LA SUITE DE L'EXERCICE SUR LA PROCHAINE DIAPOSITIVE…

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.29 Exercice 2 (2/2)

Exercice 2 (suite)

- Ajouter un fichier de type tableur dans votre projet
- Que se passe t'il dans l'onglet "Git" de RStudio ? Avec la commande `git status` ?
- Modifier le fichier `.gitignore` pour faire en sorte que Git ignore ce fichier tableur.
- Ajouter vos changements sur `.gitignore` à l'index et faire un nouveau commit

- Ajouter un nouveau script `Analyse.R` qui contient uniquement la ligne `library(dplyr)`
- Ajouter vos changements à l'index et faire un nouveau commit

- Consulter l'historique de votre projet avec RStudio et le terminal

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4 Travailler avec une forge

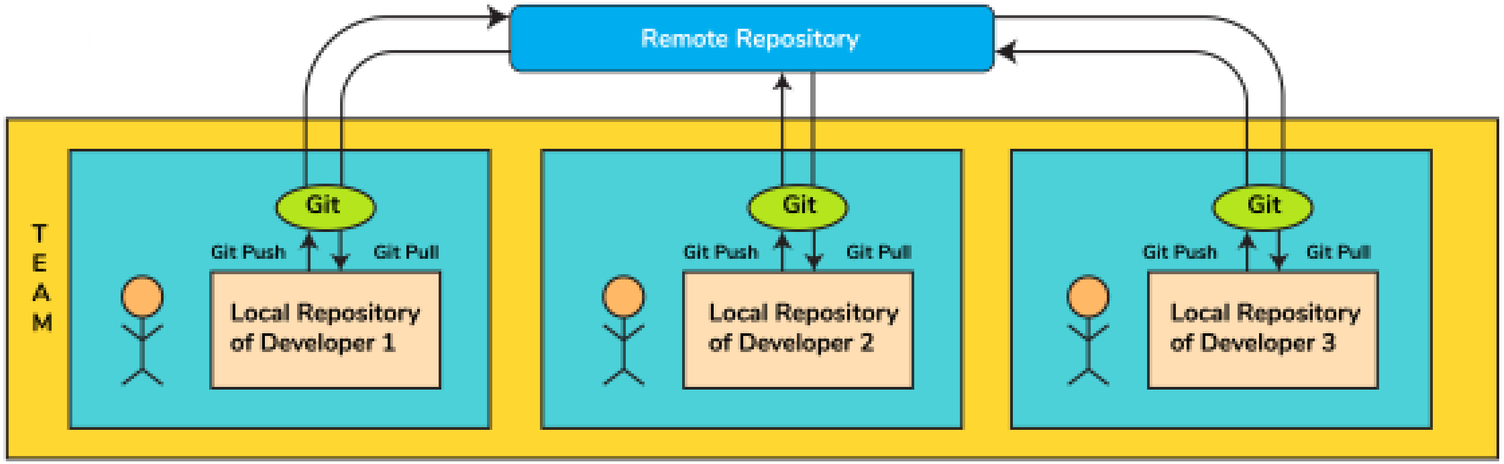
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.1 Git - un modèle distribué

- **Dépôt local** : dépôt de l'agent sur son espace de travail (sous Cerise ou en local par ex.)
- **Dépôt distant (remote)** : dépôt correspondant situé sur la forge.

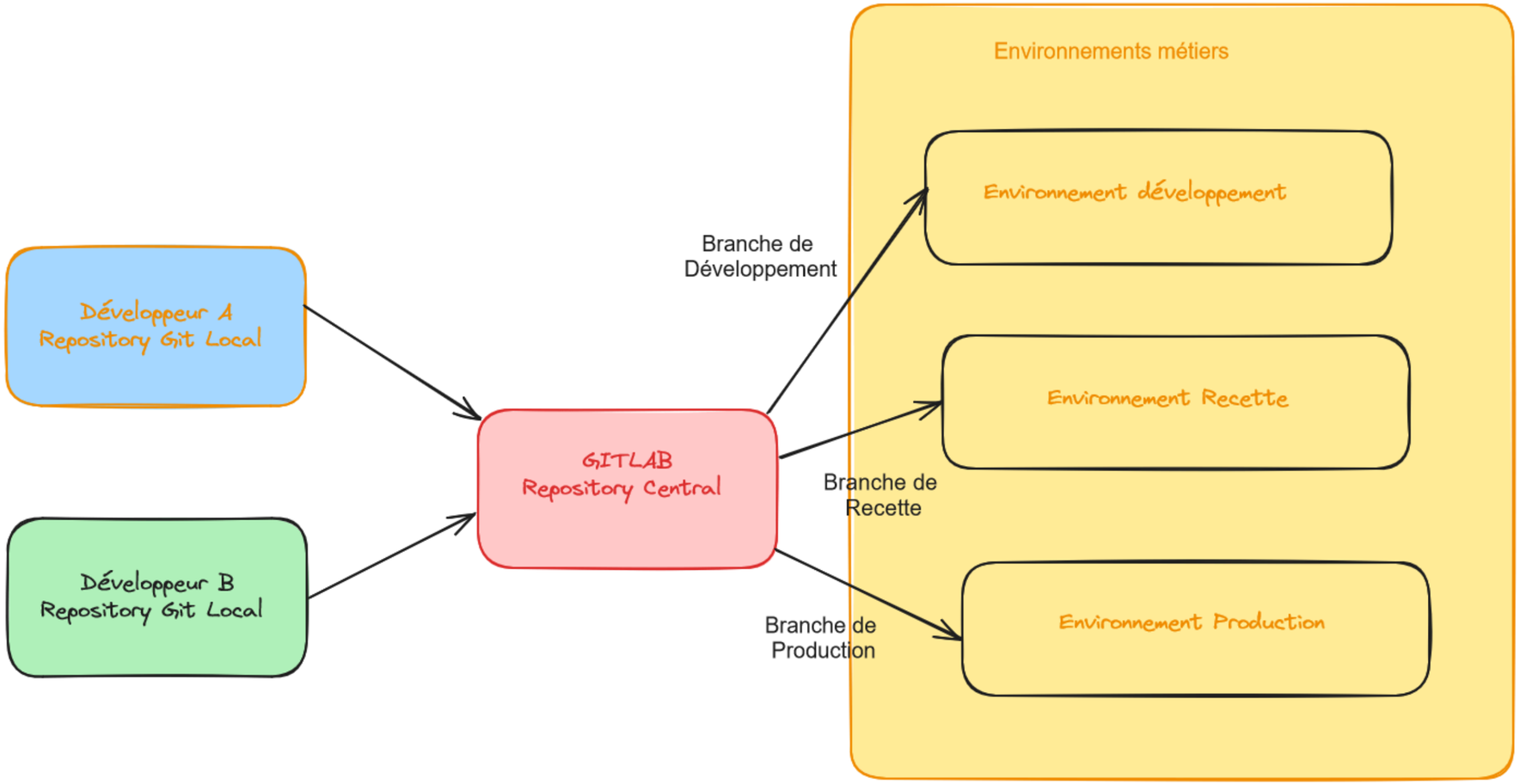
> **Convention**
>
> Par défaut, le dépôt distant porte l'alias `origin`



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.2 Architecture générale



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.3 C'est quoi une forge ?

Définition Wikipédia :

En informatique, une forge est un système de gestion et de maintenance collaborative de texte.

C'est un espace où on peut archiver du code informatique (R mais pas que bien sûr) qui offre des fonctionnalités supplémentaires à Git comme :

- la **gestion des tickets** (issues) ;
- les **merge requests ou pull requests** ;
- **un affichage** plus agréable du dépôt **avec une interface web** ;
- la possibilité d'ajouter **un wiki** ;
- **l'intégration et le déploiement continus**…

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.4 Exemples de forges

Les plus connues : Gitlab, Github, Bitbucket…

Une forge peut être **interne** ou **externe** à un ministère.

- URL de la forge interne du MAASA : https://gitlab.forge.agriculture.rie.gouv.fr/
- URL de la forge interne du MTECT : https://gitlab-forge.din.developpement-durable.gouv.fr/
- URL du SSM Agriculture (SSP) sur Github : https://github.com/SSM-Agriculture
- URL du MTECT sur Github : https://github.com/MTES-MCT

À retenir !

- Les forges externes ont vocation à héberger les projets open-source. Il s'agit donc d'être très vigilant sur le code déposé sur ces forges

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.5 Dialoguer avec Gitlab

Pour travailler avec Gitlab, il faut mettre en place un mode d'authentification.

2 modes principaux existent :

- SSH
- HTTPS

Au MAASA, **une procédure de configuration existe et a été validée en mars 2024** pour établir la connexion entre Cerise et la forge gitlab interne. Celle-ci est basée sur le mode HTTPS et utilise le credential helper de Git qui évite d'avoir à re-saisir son mot de passe (ou token) à chaque action.

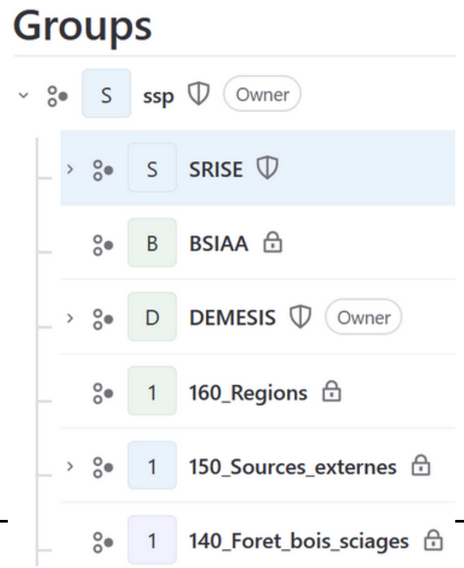
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.6 Les groupes dans les forges Gitlab

- Une forge Gitlab est structurée par **groupe**.
- Chaque **groupe** est constitué de **projets** (les fameux dépôts distants).
- Chaque **groupe** est associé à des utilisateurs dont chacun dispose **d'un niveau de permissions** sur les projets du groupe et sur le groupe lui-même.

Aperçu des groupes au MAASA/SSP :



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.7 Permissions et rôles dans Gitlab

- Les **permissions** se traduisent par plusieurs **rôles** qui sont détaillés dans la documentation de Gitlab.

Ce qu'il faut retenir :

Rôle	Permissions
Owner	Les propriétaires ont un contrôle total sur le projet ou le groupe, y compris la gestion des membres et la suppression.
Maintainer	Les mainteneurs ont des droits similaires aux propriétaires, mais n'ont pas accès à l'intégralité des paramètres sensibles du projet/groupe.
Developer	Les développeurs ont des droits d'écriture sur le code, peuvent créer des branches et fusionner des Merge Requests.
Planner	Les planificateurs se concentrent sur la gestion : ils

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.8 Bonnes pratiques avec les forges

Voici ci-dessous **quelques bonnes pratiques** (non exhaustives) à ajouter à un projet R **lorsque celui-ci est amené à être partagé sur une forge** (Gitlab ou Github ou autre).

Dans l'idéal votre projet doit contenir :

- Un fichier `README.md` : un document rédigé en **markdown** qui inclue notamment des informations sur :
  - Ce que le projet fait
  - Son emplacement sous Cerise
  - Ses pré-requis et son utilisation/installation (par exemple s'il s'agit d'une application Shiny, les instructions de lancement ou s'il s'agit d'un package R, les instructions d'installation…)
  - Ses principales fonctionnalités
  - L'identité de l'équipe qui maintient le dépôt
- Un fichier `.gitignore` (voir **cette diapo**)
- Un fichier `CHANGELOG.md` ou `NEWS.md` : lui aussi rédigé avec markdown, il permet de suivre les principales modifications apportées au projet par les développeurs. Il est très utile en association avec les **tags**.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.9 Le système de ticketing (issues)

Une issue est une tâche à effectuer : il peut s'agir tout autant de correctifs à apporter au projet que de fonctionnalités à ajouter.

Ouvrir une issue depuis Gitlab permet de :

– **Discuter un point** : il est possible d'échanger à plusieurs dans une issue pour définir comment la traiter.
– **Assigner une tâche à une personne** : lorsqu'on crée une issue, on peut assigner la tâche à une ou plusieurs personnes. Elle s'affiche alors sur le tableau de bord.
– **Définir une échéance, classer une issue avec un label** : le tableau de bord permet alors d'avoir une vision complète des issues en cours.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

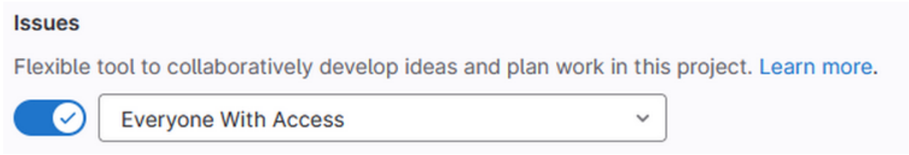
Liberté
Égalité
Fraternité

4.10 Activer les issues sur la forge interne du MAASA

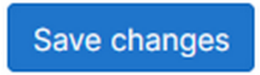
Par défaut, les issues ne sont pas activées à la création d'un dépôt sur la forge Gitlab du MAASA.

Pour les activer, il faut :

- Cliquer sur `Settings` dans le bandeau de gauche puis `General`
- Cliquer sur le bouton **Expand** à côté de `Visibility, project features, permissions`
- Activer les issues :



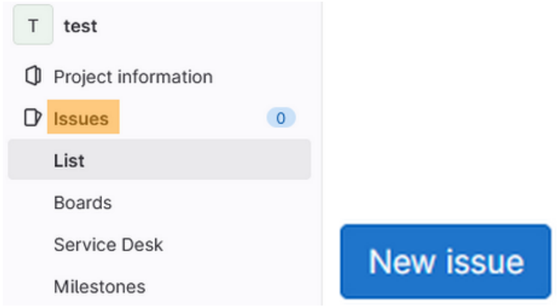
- Enregistrer les changements :



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.11 Créer une issue



Puis renseigner les informations concernant l'issue.



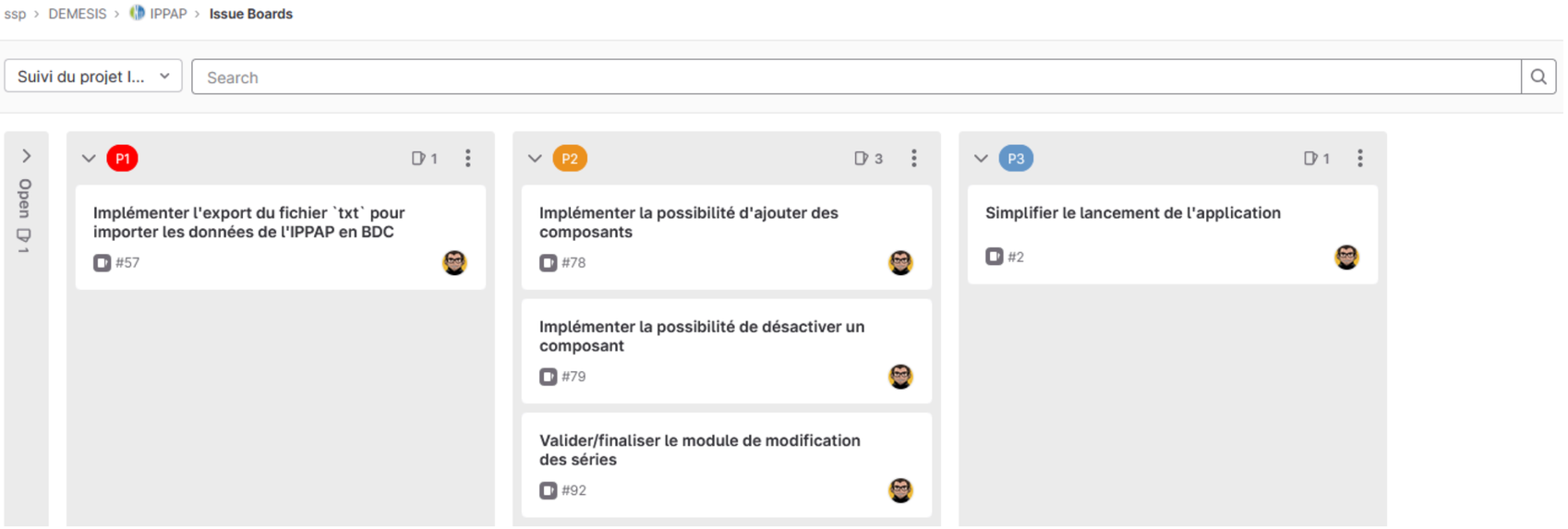
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.12 Afficher les issues sous forme de tableau (board)

Si vous utilisez **des labels** pour classer vos issues, Gitlab propose un affichage sous forme de tableau de bord qui permet de faire **un suivi de projet sous forme de tâches (Kanban)**.

Exemple :



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

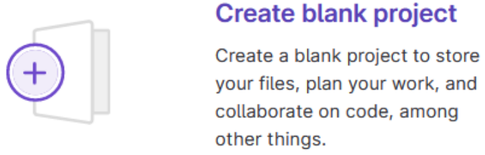
4.13 Lier un dépôt local à un dépôt distant (1/3)

1ère étape commune à toutes les méthodes : création d'un dépôt vide sur la forge Gitlab

- Cliquez sur le bouton "New project" situé en haut à droite :



- Choisir "blank project" :

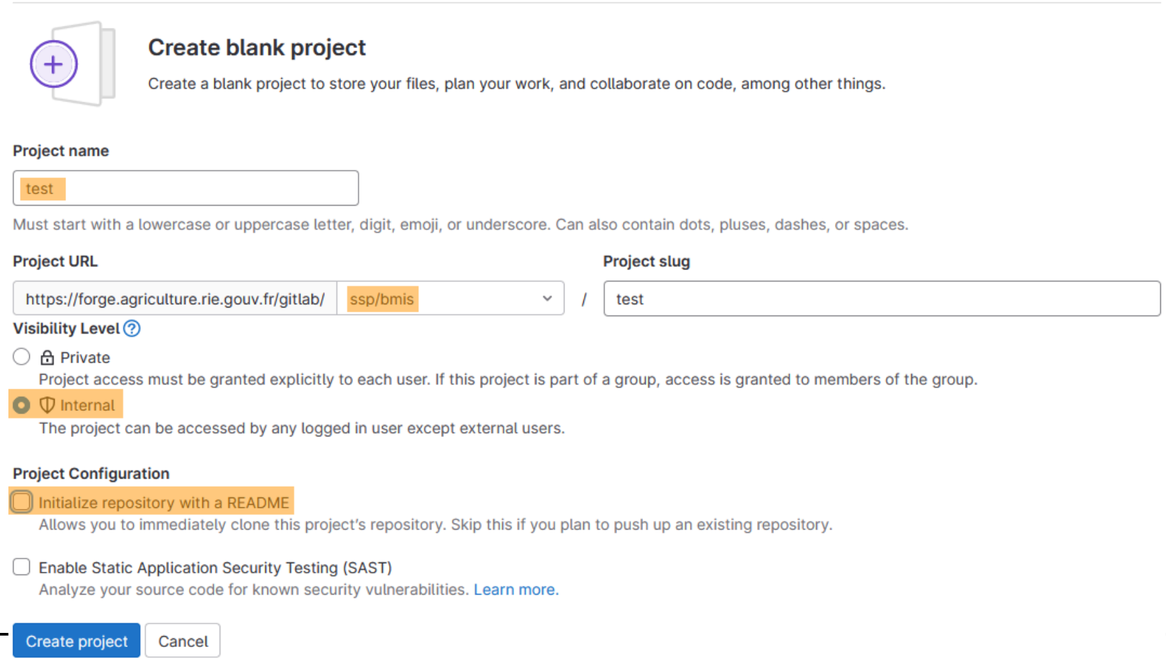


MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.14 Lier un dépôt local à un dépôt distant (2/3)

- Renseigner le nom du projet, éventuellement une courte description, l'espace d'appartenance du projet dans la forge, son niveau de visibilité et penser à décocher l'option "Initialize repository with a README". Puis cliquer sur "Create Project".



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.15 Lier un dépôt local à un dépôt distant (3/3)

La méthode à utiliser est différente selon les cas :

1. **Vous voulez créer un nouveau répertoire** dans votre working directory
2. **Vous avez déjà un répertoire** dans votre working directory

À retenir

Pour vérifier qu'un dépôt local est lié à un dépôt distant, la commande du terminal `git remote -v` est très utile car elle renvoit l'URL du dépôt distant s'il existe.

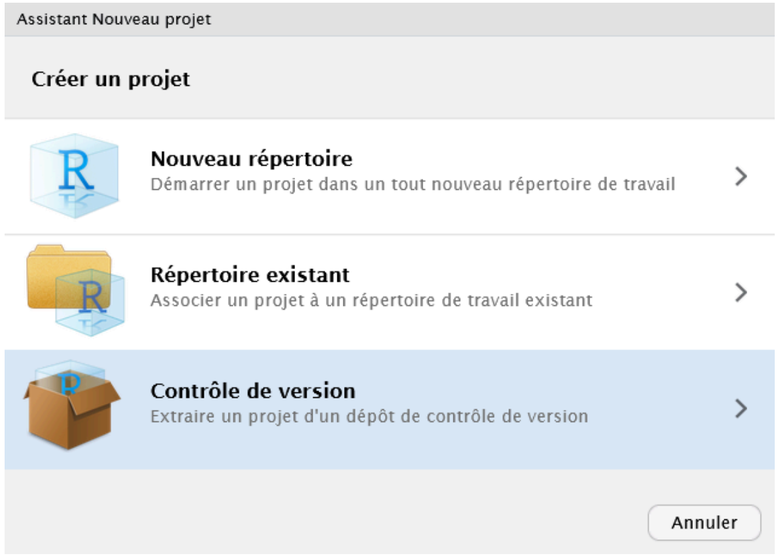
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.16 Lier un dépôt local à un dépôt distant avec RStudio (1/5)

CAS 1 : création d'un nouveau répertoire

Créer un nouveau projet, puis choisir "Contrôle de version" :



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.17 Lier un dépôt local à un dépôt distant avec RStudio (2/5)

Puis choisir “Git” :



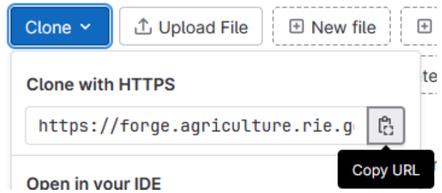
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.18 Lier un dépôt local à un dépôt distant avec RStudio (3/5)

Puis renseigner :

- L'URL du dépôt distant (voir ci-contre comment la récupérer) ->
- Le nom du répertoire du projet
- Le chemin du dépôt local dans votre espace de travail





MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.19 Lier un dépôt local à un dépôt distant avec RStudio (4/5)

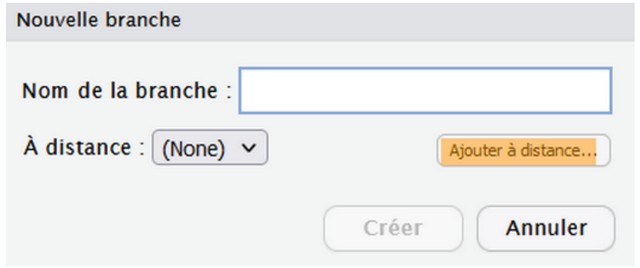
CAS 2 : un répertoire existe déjà

Dans ce cas, l'interface de RStudio est mal pensée et **il est beaucoup plus intuitif d'utiliser le terminal** (voir ici)

La fonctionnalité d'ajout d'un dépôt distant est accessible en cliquant sur le bouton "Nouvelle branche" :(



Puis cliquer sur le bouton "Ajouter à distance" :



Entrer l'alias origin et l'URL du dépôt distant :



Puis cliquer sur "Ajouter" puis "Annuler" sur l'écran précédent (on ne veut pas créer de nouvelle branche ici)

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.20 Lier un dépôt local à un dépôt distant avec RStudio (5/5)

Avec la procédure présentée dans le cas 2, les boutons “Git-pull” et “Git-push” apparaîssent en grisé et ne sont pas disponibles pour l'utilisateur :



Pour dégriser ces boutons, **le terminal est indispensable** pour taper la commande :

```
git push -u <alias> <nom-branche>
```

Puis en cliquant sur le bouton de rafraîchissement du l'onglet “Git” 

> **Note**
>
> Pour information, le `-u` de la commande ci-dessus est nécessaire car dans le cas 2, aucune branche n'existe dans le dépôt local. Cette instruction permet simplement de créer une branche locale qui suit la branche distante existante sur le dépôt distant.
>
> ```
> To https://forge.agriculture.rie.gouv.fr/gitlab/damien.dotta/formation.git
>  * [new branch]      main -> main
> ```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.21 Lier un dépôt local à un dépôt distant avec terminal (1/2) …

CAS 1 : création d'un nouveau répertoire

- Lancer le terminal dans le répertoire où on souhaite créer le nouveau dossier correspondant au dépôt distant (rappel sur **comment faire**)
- Taper la commande `git clone <URL-depot-distant>`

Par exemple :

```
git clone https://gitlab.forge.agriculture.rie.gouv.fr/ssp/bac-a-sable/test-formation-git
```

- Ouvrir le nouveau répertoire et commencer à y travailler comme d'habitude en y créant un projet, des scripts…

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.22 Lier un dépôt local à un dépôt distant avec terminal (2/2)

CAS 2 : un répertoire existe déjà

- Lancer le terminal dans le répertoire que l'on souhaite lié au dépôt distant (rappel sur **comment faire**)
- Taper la commande `git remote add origin <URL-depot-distant>`

Par exemple :

```
git remote add origin https://gitlab.forge.agriculture.rie.gouv.fr/ssp/bac-a-sable/test-formation-git
```

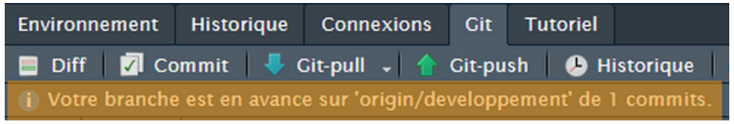
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.23 Pousser les changements sur la forge avec RStudio

Après avoir ajouté vos changements à l'index et fait vos commits, vous pouvez **"pousser"** ces modifications sur la forge avec RStudio.

Dans l'exemple ci-dessous, après avoir committer une fois, RStudio me prévient dans l'onglet "Git" que je suis sur la branche `developpement` d'un commit :



Pour pousser ce commit sur le dépôt distant, il suffit simplement de cliquer sur le bouton 

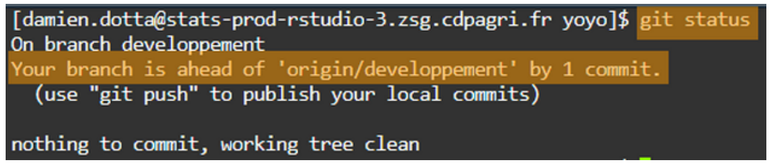
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

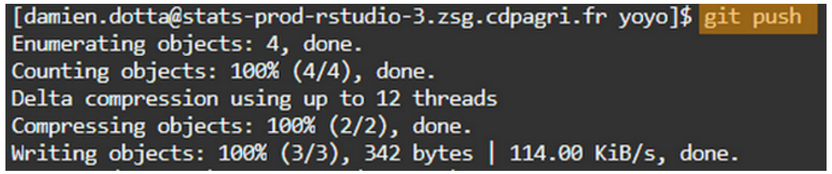
4.24 Pousser les changements sur la forge avec le terminal

Voilà comment pousser vos modifications sur la forge avec le terminal :

- Taper la commande `git status` pour récupérer l'information de la situation de notre dépôt local par rapport au dépôt distant.



- Taper la commande `git push` **pour pousser ce commit sur le dépôt distant**.



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.25 Utilisation des instructions `git push` et `git pull`

Dans les slides précédentes, une utilisation succinte des instructions `git push` et `git pull` a été faite.

Pour être sûr de maîtriser vos interactions entre le dépôt local et le dépôt distant, il est recommandé d'utiliser la syntaxe plus complète sur le modèle suivant :

- `git push <alias> <nom-branche>`
- `git pull <alias> <nom-branche>`

Exemples :

- `git push origin developpement`
- `git pull origin production`

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.26 Exercice 3

Exercice 3

Reprendre l'exercice 2

- Aller sous dans la forge Gitlab dans le groupe « ssp/BAC-A-SABLE »
- Créer un dépôt vide dans ce groupe au format "prenom-nom-formation-git" dans la forge Gitlab
- Lier votre projet R à ce dépôt distant
- Poussez vos commits déjà effectués sur le dépôt distant

- Apporter des changements dans un de vos scripts R (sauf `Analyse.R`)
- Ajouter vos changements à l'index, faire un nouveau commit et pousser sur le dépôt distant

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.27 Récupérer les changements en local avec RStudio

Pour récupérer sur le dépôt local des changements qui ont été poussés sur le dépôt distant avec RStudio.

Exemples de cas d'usage :

- Si un membre de l'équipe projet a poussé des modifications avant vous sur le dépôt distant.
- Si vous voulez récupérer les dernières modifications apportées à la branche distante suite à une `merge request` (voir plus loin)

Pour récupérer les commits depuis le dépôt distant, il suffit simplement de cliquer sur le bouton 

Dans le cas d'un travail en équipe, il s'agit d'une opération à faire très fréquemment.
Git vous empêchera de pousser vos modifications sur le dépôt distant si vous n'avez pas au préalable récupérer toutes les dernières modifications qui s'y trouvent.

Message qui s'affiche dans ce cas :

```
>>> git push origin HEAD:refs/heads/developpement
To https://forge.agriculture.rie.gouv.fr/gitlab/damien.dotta/test.git
 ! [rejected]        HEAD -> developpement (fetch first)
error: failed to push some refs to 'https://forge.agriculture.rie.gouv.fr/gitlab/damien.dotta/tes
t.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.28 Récupérer les changements en local avec le terminal

- Taper la commande `git pull` **pour récupérer les derniers commits depuis dépôt distant.**

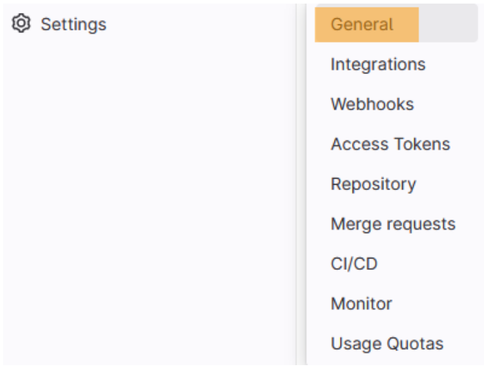
```
[damien.dotta@stats-prod-rstudio-3.zsg.cdpagri.fr yoyo]$ git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 343 bytes | 5.00 KiB/s, done.
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

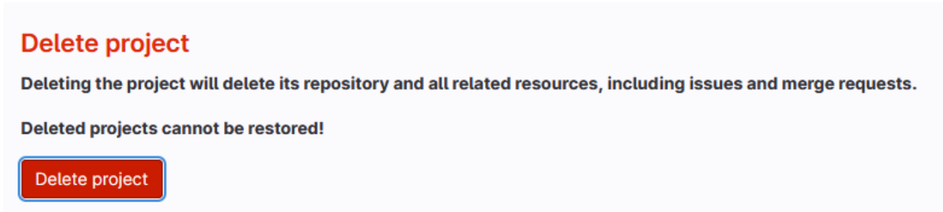
Liberté
Égalité
Fraternité

4.29 Supprimer un dépôt dans Gitlab

- Si vous êtes owner du projet :



Descendre en bas de la page puis :



- Sinon, contacter la BAL : assistance.si-stat.sg@agriculture.gouv.fr.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.30 Créer des tags avec le terminal

Les tags sont des étiquettes qui pointent vers des points spécifiques dans l'historique de Git.
Ils sont très utiles pour étiqueter les versions de vos projets (v0.1, v0.2…).

Pour créer un tag v0.1 sur le dernier commit :

```
git tag -a v0.1 -m "Message de tag"
```

Pour créer un tag v0.1 a posteriori (avec l'ID du commit) :

```
git tag -a v0.1 0af8bfd9 -m "Message de tag"
```

Pour partager les tags sur le dépôt distant :

```
git push origin --tags
```

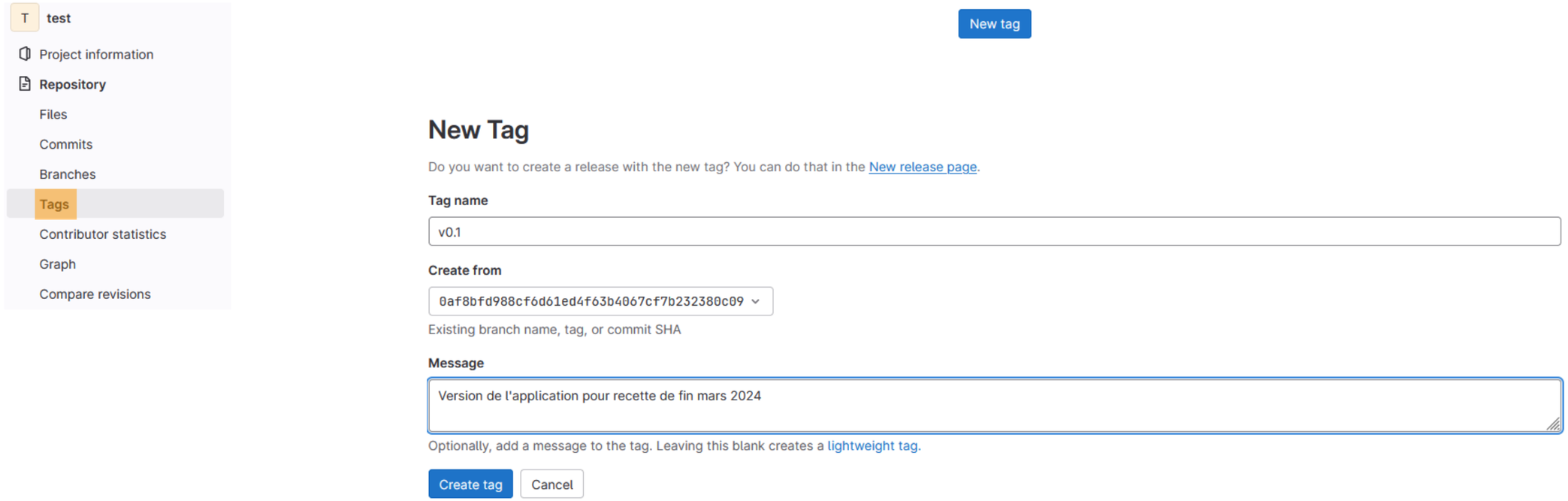
Pour lister les tags :

```
git tag
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.31 Créer des tags avec gitlab



Il reste à faire un `git pull` pour récupérer le nouveau tag dans votre working directory.

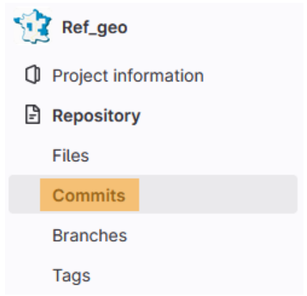
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

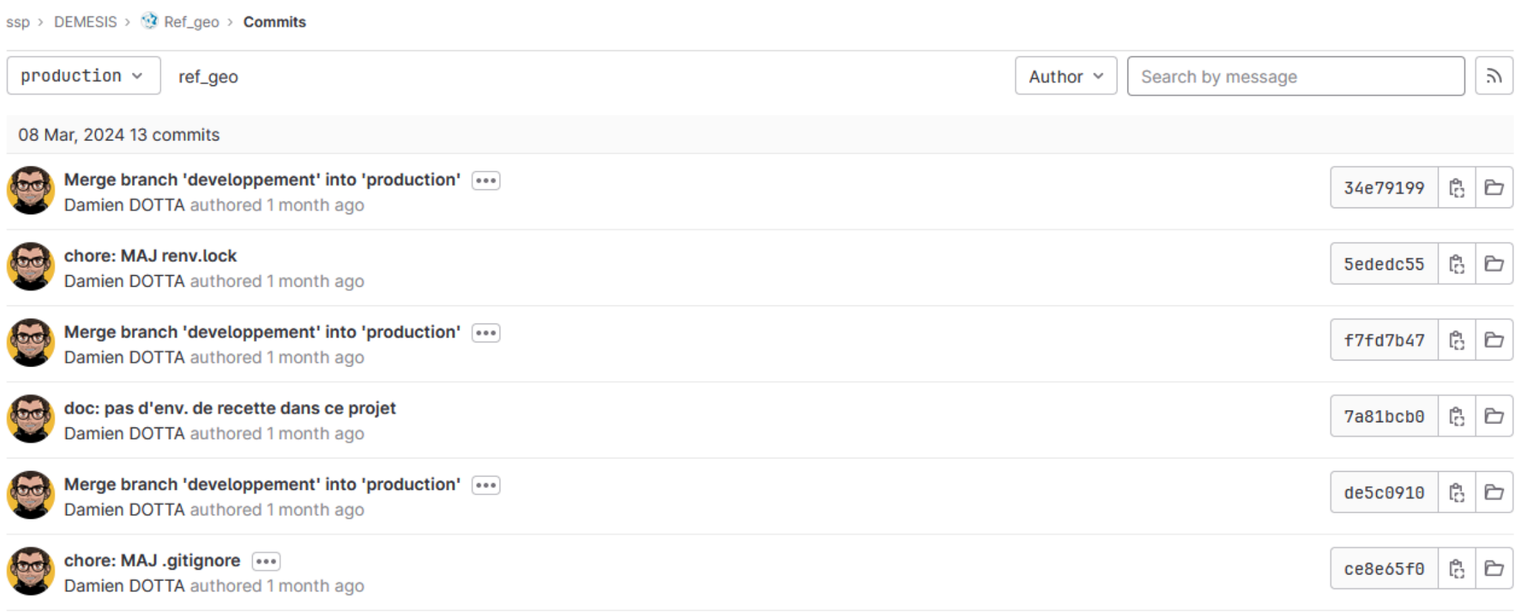
Liberté
Égalité
Fraternité

4.32 Revenir en arrière avec Gitlab (1/3)

L'interface de Gitlab permet de facilement se déplacer dans l'historique d'un projet versionné.

Par exemple :



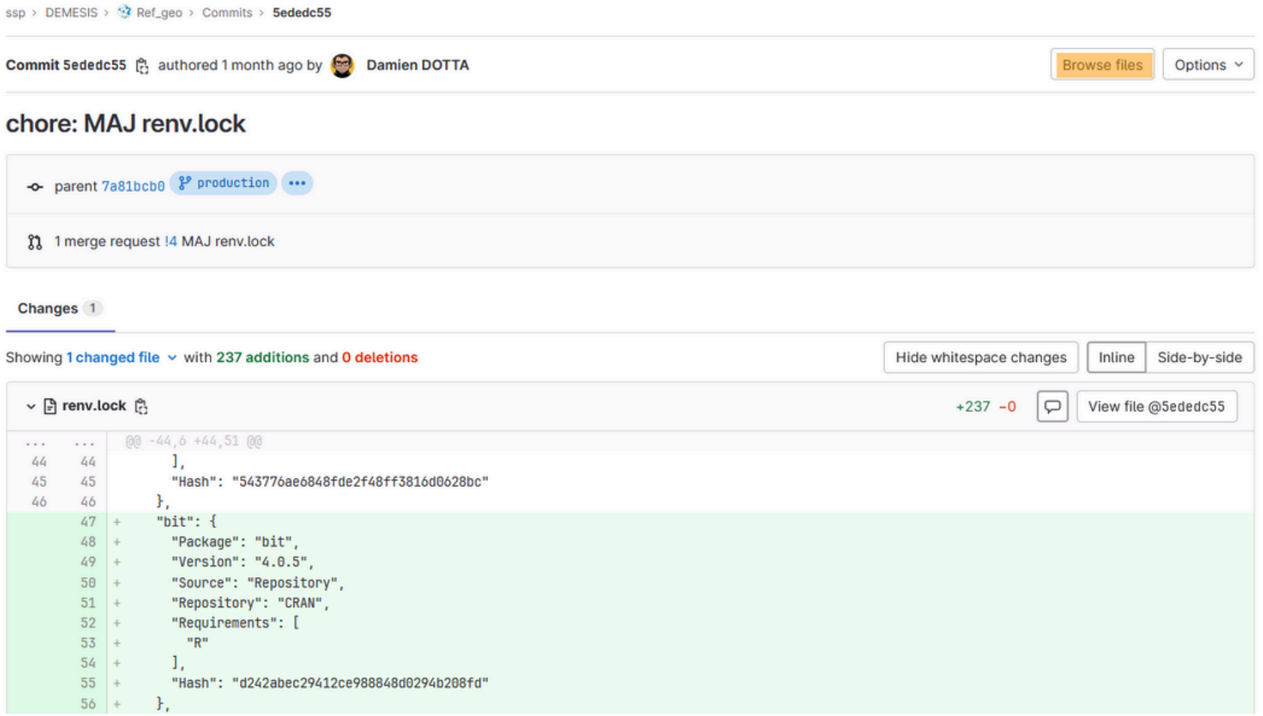


MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.33 Revenir en arrière avec Gitlab (2/3)

Cliquer sur le commit d'intérêt et dans la fenêtre qui en détaille le contenu, cliquer sur "Browse file" :

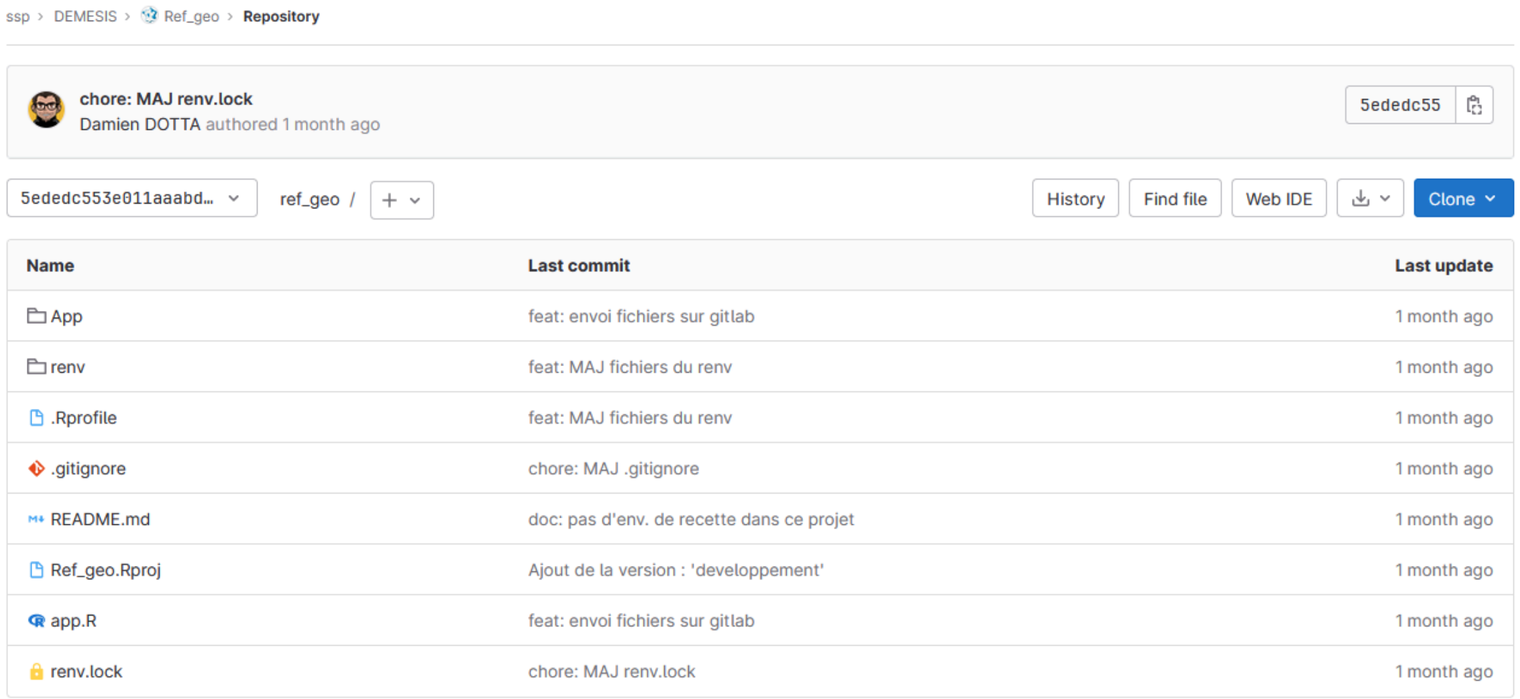


MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.34 Revenir en arrière avec Gitlab (3/3)

On accède ainsi à l'état du projet tel qu'il était au moment du commit d'intérêt.



Remarque : des manipulations quasi-identiques permettent de faire la même chose avec les tags.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

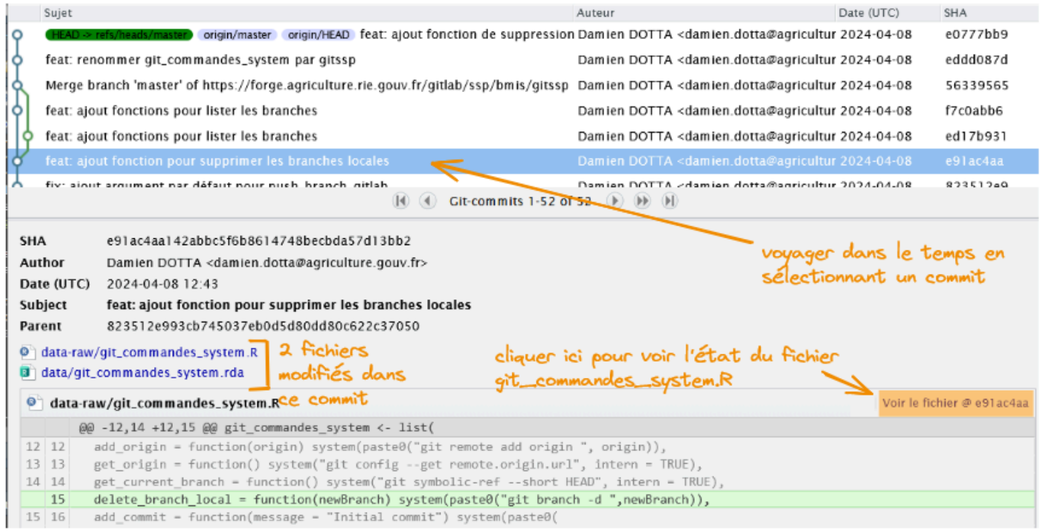
Liberté
Égalité
Fraternité

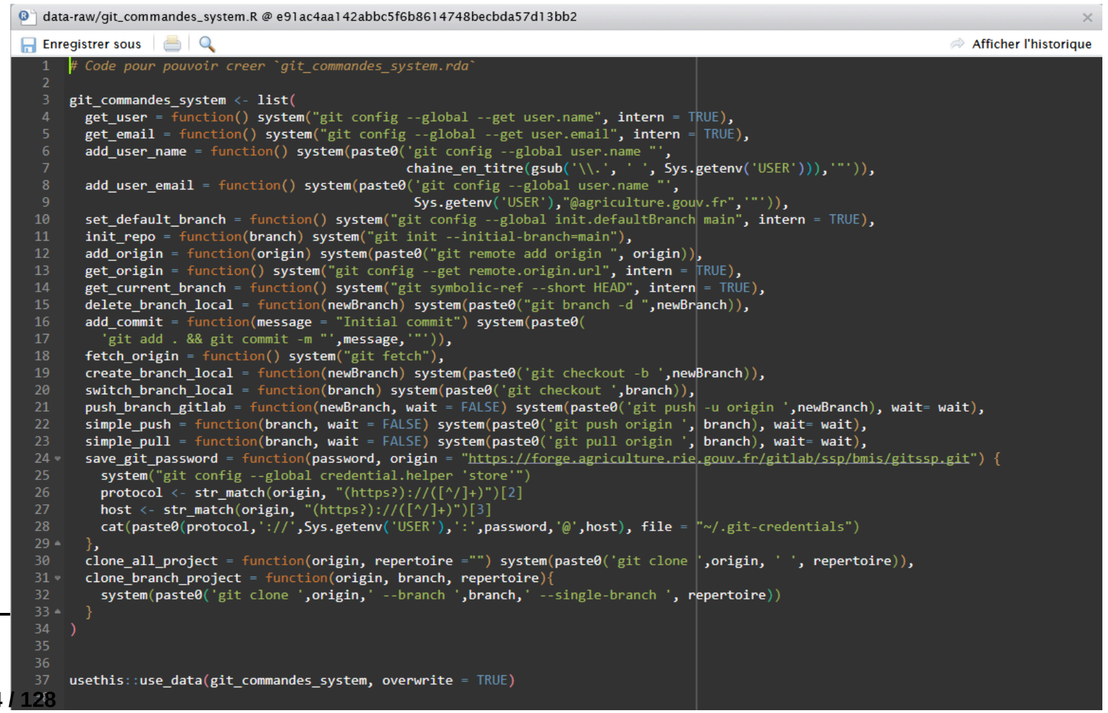
4.35 Revenir en arrière avec RStudio

Avec RStudio, en cliquant sur le bouton “Historique” : Historique

On accède aux différentes versions des fichiers qui ont été modifiés au fil du temps

Remarque : avec RStudio il n’est pas possible de revenir en arrière sur l’ensemble du projet





MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.36 Revenir en arrière avec le terminal

Pour revenir à la version du tag v0.1 :

```
git checkout v1.0
```

Pour revenir à la version du commit 0af8bfd9 :

```
git checkout 0af8bfd9
```

Pour remonter de 4 commits dans le passé :

```
git checkout nom-de-branche~4
```

Pour revenir au présent :

```
git checkout nom-de-branche
```

> **Note**
>
> Les instructions Git ci-dessus peuvent aussi se décliner sur un fichier spécifique.
> Par exemple avec la structure suivante pour un commit et un fichier particulier : `git checkout commitID file-name`

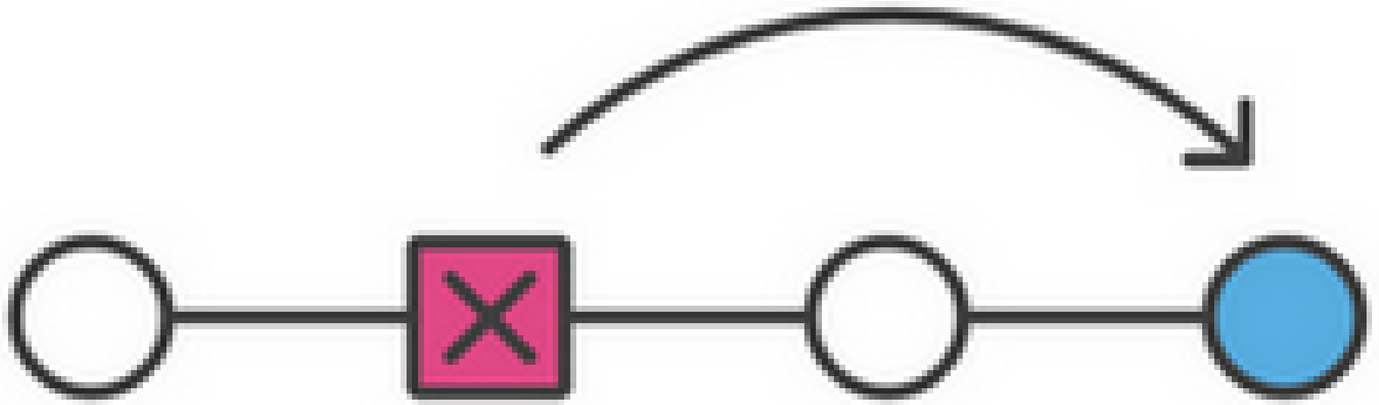
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.37 Inverser un commit

La commande `git revert SHA-commit` permet d'inverser/de défaire ce qui avait été fait au moment du commit d'intérêt.

Attention, le `git revert` crée un nouveau commit (d'inversion c'est-à-dire que les lignes ajoutées seront supprimées et les fichiers supprimés seront recréés…).



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.38 Si vous ne deviez retenir que quelques commandes ! À retenir !

Action	R	>_
Vous ajoutez vos fichiers à l'index	Cochez les fichiers	`git add`
Vous committez vos changements	Appuyez sur le bouton "Commit" et vous renseignez votre message de commit	`git commit - m "Votre message"`
Vous envoyez vos commits en local sur le serveur distant	Appuyez sur le bouton "Push"	`git push origin <nom-de-branche>`
Supplément si vous travaillez à plusieurs :		
Récupérer les éventuels commits de vos collègues	Appuyez sur le bouton "Pull"	`git pull origin <nom-de-branche>`

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.39 Autres instructions Git

Il y a encore pleins de concepts et d'instruction Git très utiles que vous découvrirez (ou pas) :

- Le **fork** : travailler avec une copie d'un dépôt existant
- **git cherry-pick** : sélectionner et appliquer un seul commit d'une branche à une autre
- **git stash** : mettre de côté les modifications d'un répertoire de travail
- **git blame** : montrer la révision et l'auteur qui ont modifié en dernier chaque ligne d'un fichier
- **git rebase** : réappliquer une série de commits d'une branche sur une autre branche
- **git bisect** : trouver par recherche binaire la modification qui a introduit une erreur

  …

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.40 Exercice 4

Exercice 4

Reprendre l'exercice 3

- Parcourir l'historique de votre dépôt distant avec Gitlab (les solutions avec RStudio et le terminal ont été abordées lors de l'exercice 2).
- Faire apparaître les différences entre deux versions consécutives du projet
- Afficher une version passée du projet (par exemple son état au 2ème commit)

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

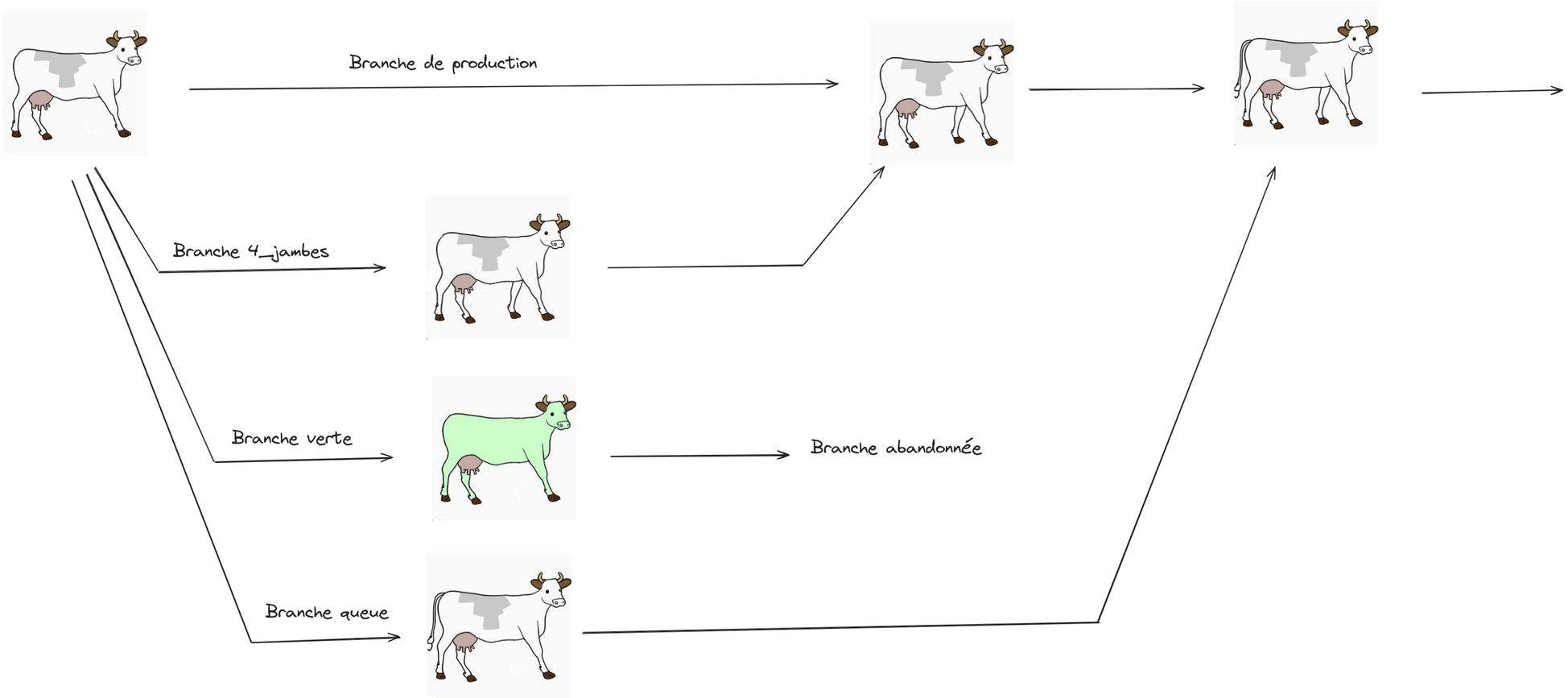
Liberté
Égalité
Fraternité

5 Les branches

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.1 Définition des branches



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.2 Quand créer une branche ?

On crée une branche pour :

- Continuer le développement de manière isolée sans altérer le reste du dépôt
  - Pour **corriger un bug**
  - Pour **implémenter une nouvelle fonctionnalité**
  - Pour **refactorer le code**…
- Isoler les différentes étapes du développement (une branche = un environnement)

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.3 Noms des branches

Concernant le noms des branches, une bonne pratique est d'utiliser la structure suivante : `<categorie>-<nom>`

Catégorie	Signification
hotfix	Pour résoudre rapidement des problèmes critiques
bugfix	Pour résoudre des erreurs/bugs
feature	Pour ajouter/supprimer/modifier des fonctionnalités
test	Pour tester une idée expérimentale (hors issue)
issue-X	Pour faire référence à l'issue n°X

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.4 Fermer une issue à partir d'une branche

Avec les forges Gitlab et Github, la fermeture des issues peut être **automatiquement gérée via les messages de commit**.

Pour cela, vous pouvez utiliser des **mots-clés** comme (au choix) :

- `Close`
- `Fix`
- `Resolve`
- `Implement`

En précisant le `#numero-issue`.

Exemple de message de commit :

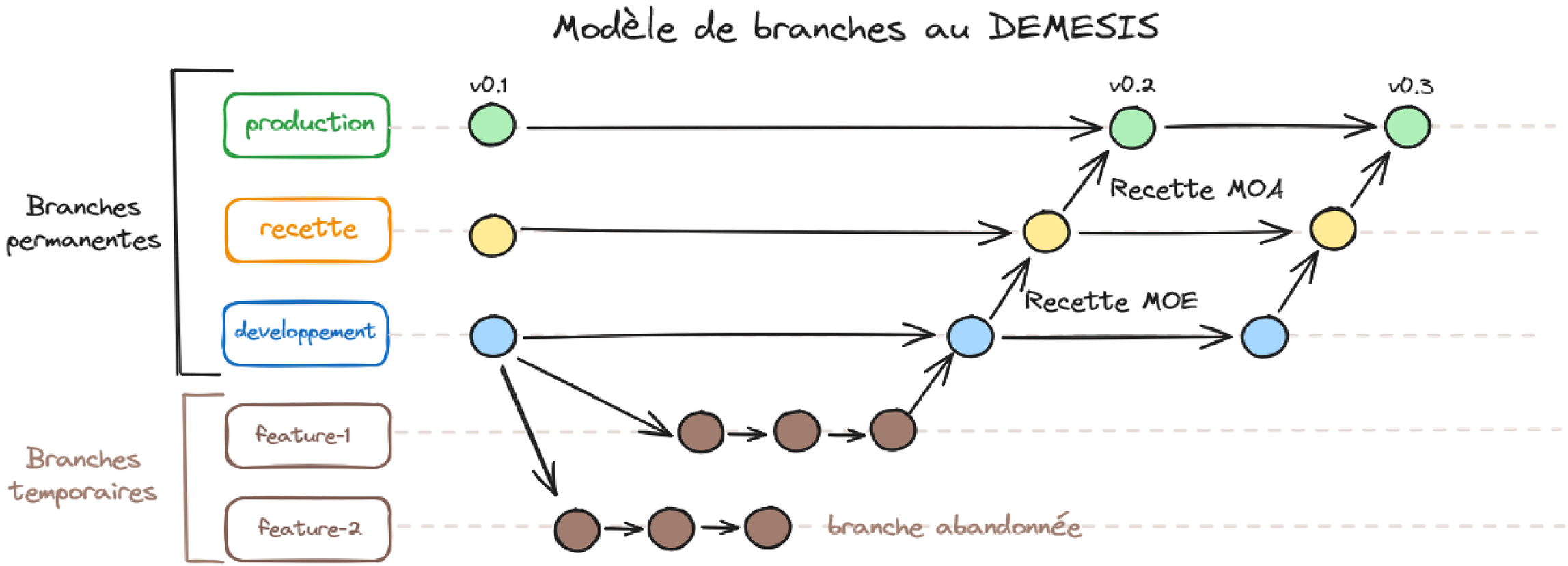
```
feat: Ajout documentation utilisateurs

Close #31, en lien avec #29
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.5 Organisation au DEMESIS



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.6 Que deviennent les branches ?

Les branches permanentes persistent tout au long de la vie du projet.

Les branches temporaires :

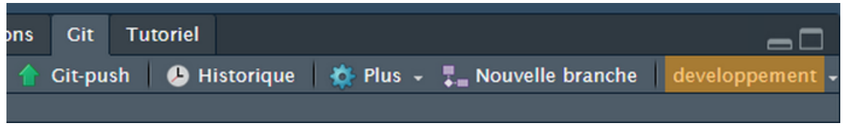
- Apparaissent et disparaissent au fil de la vie du projet
- Ont une durée de vie limitée…
- … et doivent donc être détruites (manuellement ou suite à une fusion)

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.7 Savoir sur quelle branche on se situe

RStudio indique la branche sur laquelle on se situe en haut à droite de l'onglet "Git" :



`git status` est la commande la plus simple

```
[damien.dotta@stats-prod-rstudio-3.zsg.cdpagri.fr test]$ git status
On branch developpement
Your branch is up to date with 'origin/developpement'.

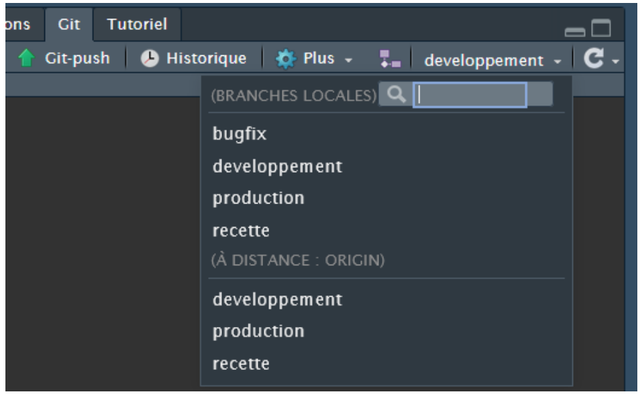
nothing to commit, working tree clean
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.8 Lister les branches

Cliquer sur la petite flèche située à côté du nom de la branche courante :



- Pour afficher uniquement les branches locales

```
git branch
```

- Pour afficher les branches locales et distantes

```
git branch -a
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.9 Liste des branches et position sur les branches

Lorsqu'on affiche la liste des branches, la branche sur laquelle on se situe est identifiée avec un astérisque * devant.

```
[damien.dotta@stats-prod-rstudio-3.zsg.cdpagri.fr test]$ git branch -a
  bugfix
* developpement
  production
  recette
  remotes/origin/developpement
  remotes/origin/production
  remotes/origin/recette
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.10 Créer une nouvelle branche avec RStudio

Pour créer une nouvelle branche avec RStudio, il faut :

- Cliquer sur le bouton “Nouvelle branche” dans l’onglet Git



- Renseigner le nom de la nouvelle branche



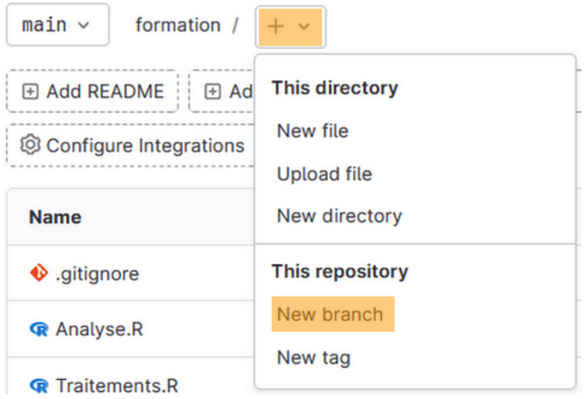
~~Cette procédure va créer la nouvelle branche~~ **~~à la fois sur le dépôt local mais aussi sur le dépôt distant.~~**

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

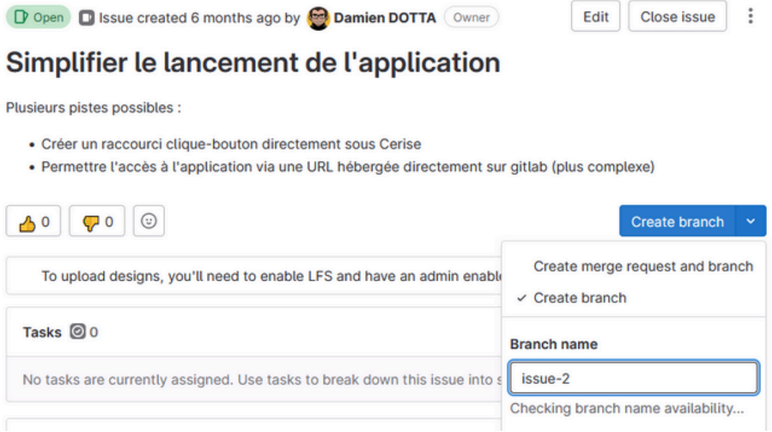
Liberté
Égalité
Fraternité

5.11 Créer une nouvelle branche à partir de Gitlab

Depuis l'interface Gitlab, on peut créer une branche depuis la page d'accueil du dépôt :



Depuis l'interface Gitlab et à partir d'une issue, on peut aussi créer une branche :



Pour récupérer cette branche distante dans votre dépôt local, on fera un `git pull` dans le terminal ou RStudio.



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.12 Créer une nouvelle branche avec le terminal

Pour créer une branche dans le dépôt local avec le terminal, les commandes `git branch` et `git checkout` sont utilisées.

Créer une nouvelle branche “bugfix” dans le dépôt local :

```
git branch bugfix
```

Créer une nouvelle branche “bugfix” dans le dépôt local et se placer dessus :

```
git checkout -b bugfix
```

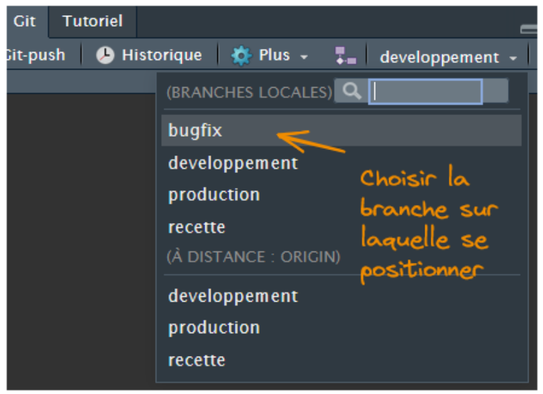
Synchroniser la nouvelle branche locale “bugfix” avec la branche distante :

```
git push -u origin bugfix
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.13 Changer de branche



Avec la commande dans le terminal :

```
git checkout nom-de-branche
```

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.14 Supprimer des branches locales



Avec l'IHM RStudio, on ne peut pas supprimer de branches locales.



Avec la commande dans le terminal :

```
git branch -d nom-de-branche
```

Important

Git empêche la suppression de branche si vous êtes positionnés dessus. Dans ce cas, il faut au préalable changer de branche avec `git checkout nom-de-branche`

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

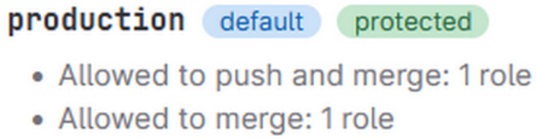
Liberté
Égalité
Fraternité

5.15 Supprimer des branches distantes

Avec l'IHM RStudio, on ne peut pas supprimer de branches distantes.

`git push origin --delete nom-de-branche` OU
`git fetch --prune`

<u>GitLab permet de protéger les branches spécifiques en empêchant les utilisateurs non autorisés de les modifier.</u>
Pour choisir les branches protégées, se rendre dans le bandeau de gauche `>settings` puis `Repository` et dérouler le menu `Branche rules`.
Puis sélectionner la ou les branches à protéger (voir **cette page** pour en savoir plus sur ce qu'il est possible de faire avec les branches protégées.)



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.16 Fusionner deux branches

Une fois les changements effectués sur votre branche de travail, on effectue **une fusion de branche** pour rapatrier ces modifications sur une branche permanente.

Dans le cadre d'un travail collectif, les fusions permettent à plusieurs membres d'une équipe de combiner leur travail.

Avec RStudio on ne peut pas fusionner 2 branches

Avec le terminal on peut fusionner 2 branches.

Par exemple :

1. On se place sur la branche de developpement : `git checkout developpement`
2. On la fusionne avec la branche de travail : `git merge branche-de-travail`
3. On pousse sur le dépôt distant : `git push origin developpement`
4. On supprime la branche de travail locale : `git branch -d branche-de-travail`

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

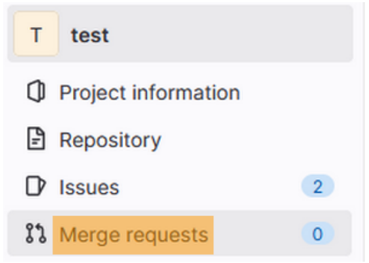
Liberté
Égalité
Fraternité

5.17 Fusionner deux branches avec Gitlab (1/3)

Il est conseillé de passer par Gitlab pour effectuer les fusions de branches.

Sous Gitlab, elle se nomment **merge request** (et **pull request** sous Github).

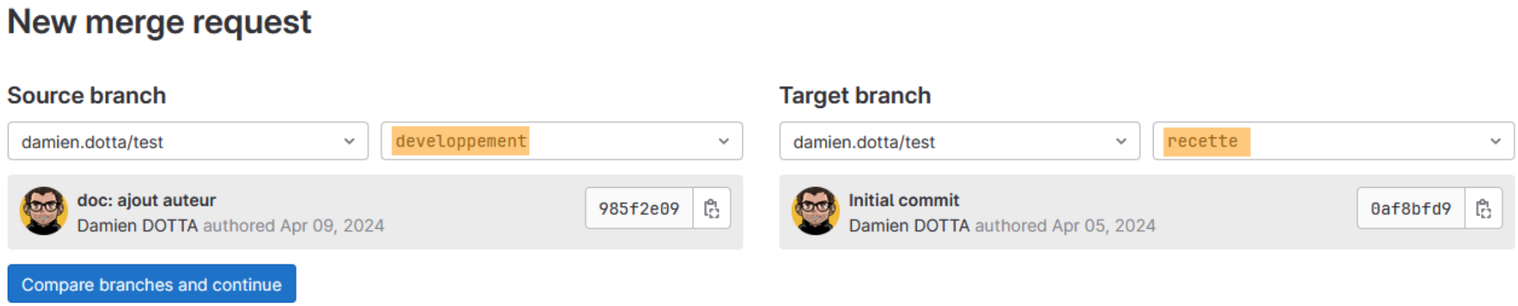
Cliquer sur "Merge requests" dans le menu de gauche :



Puis selon l'endroit par lequel vous passez, cliquer sur l'un des boutons suivants :

Create merge request ou New merge request

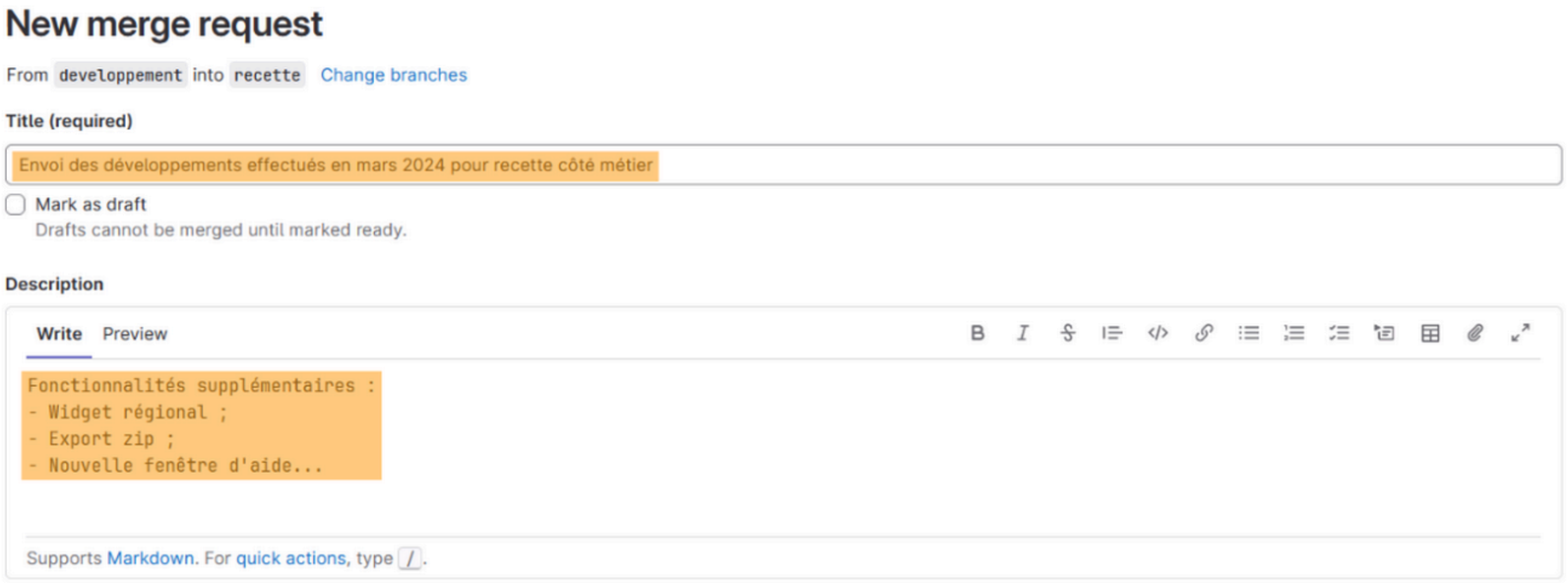
Choisir la branche source et la branche cible :



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

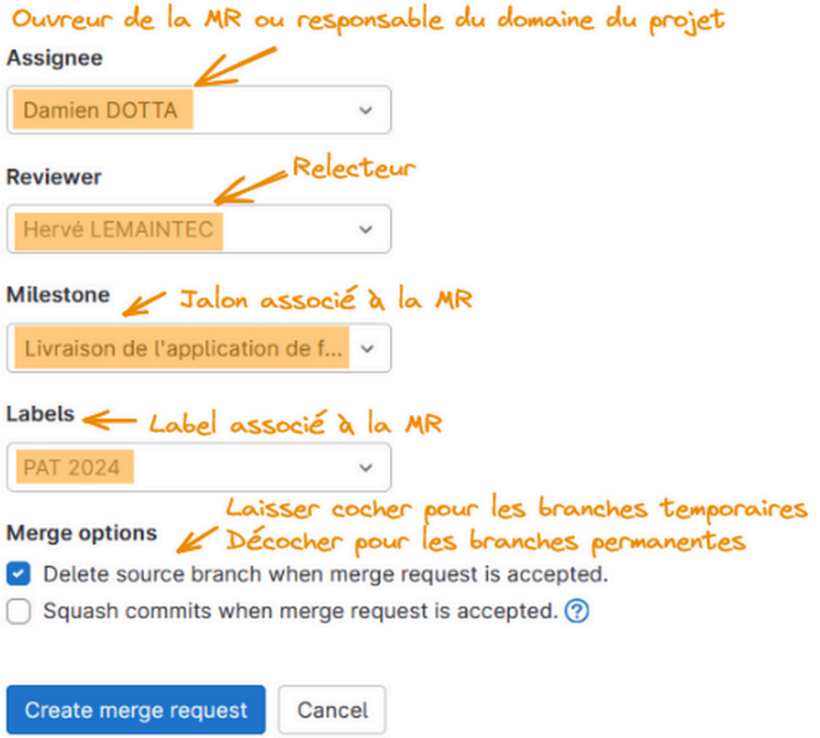
5.18 Fusionner deux branches avec Gitlab (2/3)



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.19 Fusionner deux branches avec Gitlab (3/3)

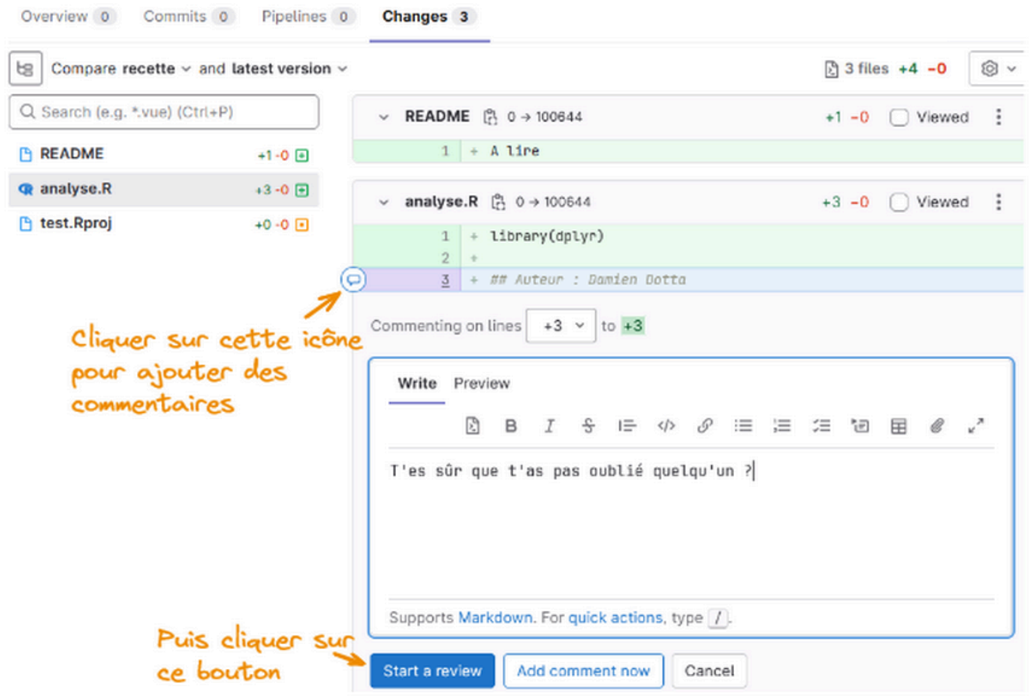


MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.20 Les relectures ou revues de code avec Gitlab (1/2)

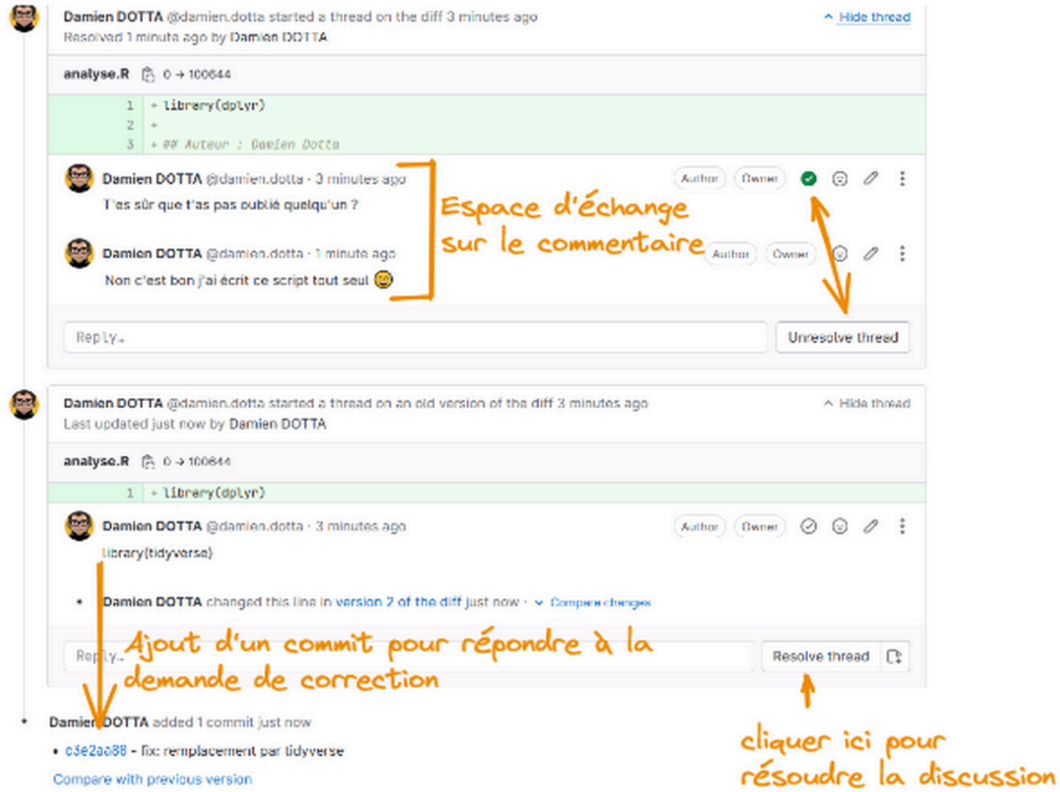
Les fusions de branches sont aussi l'occasion d'effectuer des **revues de code** entre les membres de l'équipe projet.



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.21 Les relectures ou revues de code avec Gitlab (2/2)

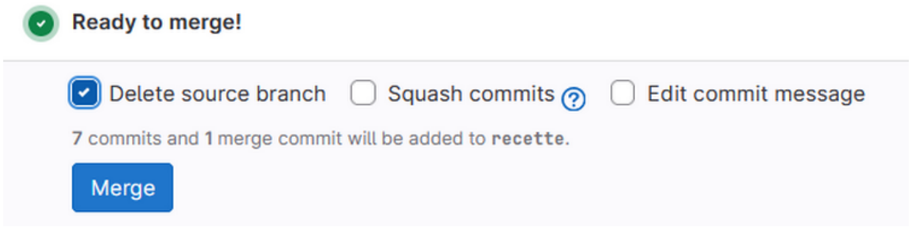


MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.22 Effectuer le merge avec Gitlab

Une fois la revue de code effectuée (pas obligatoire), appuyer sur le bouton “Merge” pour valider la fusion :



Important

Après avoir effectué la fusion sur gitlab, ne pas oublier de faire un `git pull` dans votre working directory afin de rapatrier les changements issus du merge !

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

5.23 Stratégies de fusion

Sans rentrer dans le détail, il existe 2 principales stratégies de merge dans Git :

- le `fast-forward-merge` ou `merge rapide` : utilisée principalement pour la **correction de petites fonctionnalités**
  Il a lieu lorsque le chemin entre la pointe de la branche-de-travail et la branche permanente **est linéaire**.
- le `3-way-merge` ou `merge à 3 sources` : utilisée pour l'intégration de **fonctionnalités de plus long terme**
  Il a lieu lorsque la branche permanente a été modifiée pendant les développements effectués sur la branche-de-travail.

Pour en savoir plus, consulter **cette page** très complète sur le sujet et en français.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

5.24 Définition des conflits

Les conflits surviennent généralement lorsque **deux personnes ont modifié les mêmes lignes dans un fichier**, ou si par exemple un développeur a supprimé un fichier alors qu'un autre développeur le modifiait.

Dans ces cas, **Git ne peut pas déterminer automatiquement la version correcte**

Comportement de Git lors d'un conflit :
- **Les conflits n'affectent que le développeur qui effectue le merge**
- Les autres membres de l'équipe ne sont pas conscients du conflit.
- Git marquera le fichier comme étant en conflit et bloquera le processus de merge.

=> **Il incombe alors aux développeurs de résoudre le conflit.**

Source: Atlassian

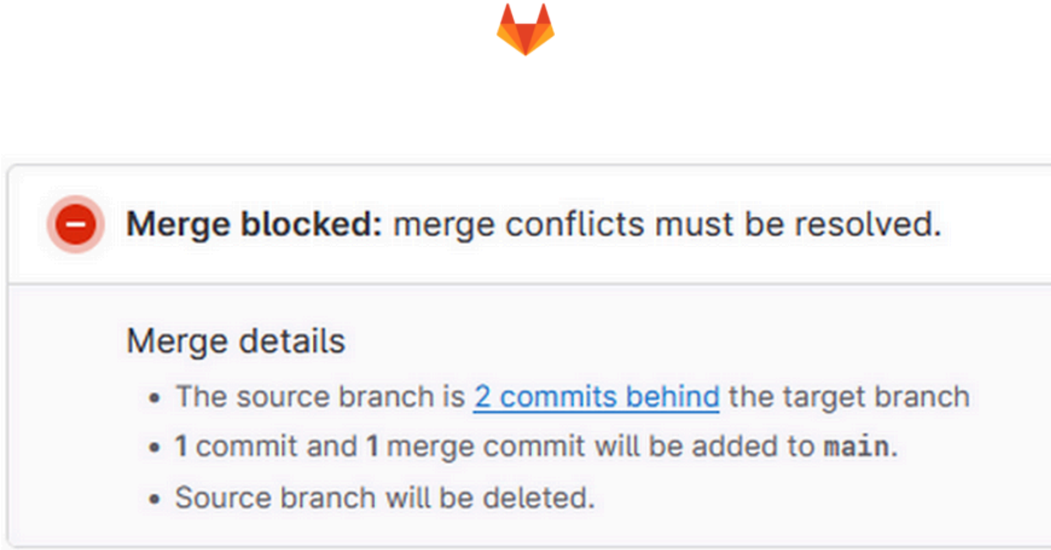
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.25 Gérer les conflits (1/2)

Que ce soit avec le terminal ou sur Gitlab, le (seul) avantage des conflits est qu'on est vite avertis :)

```
[damien.dotta@stats-prod-rstudio-3.zsg.cdpagri.fr form]$ git merge bugfix-tidyr
Auto-merging Analyse.R
CONFLICT (content): Merge conflict in Analyse.R
Automatic merge failed; fix conflicts and then commit the result.
```



Et si jamais cela ne suffisait pas, **Gitlab nous envoie un mail** pour nous dire qu'une fusion n'a pas pu avoir lieu.

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.26 Gérer les conflits (2/2)

Gitlab nous propose 2 façons de résoudre les conflits, soit localement (ce qui correspond à la procédure à suivre avec le terminal) soit directement dans l'interface de Gitlab :



- **La résolution sur Gitlab :**
  Cela signifie que l'arbitrage par l'équipe projet se fait directement sur l'interface Gitlab. Il s'agira ensuite de récupérer ces modifications sur le dépôt local.
- **La résolution locale :**
  Cela signifie que l'on rapatrie dans son working directory les fichiers tels qu'ils sont en cours lors de la fusion. Après arbitrage de l'équipe projet sur les lignes de code à retenir, il s'agit ensuite de pousser la modification sur le dépôt distant.
  => Dans ce cas, on utilisera le terminal

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.27 Délimiter les conflits

Pour délimiter la zone de conflit, Git utilise les marqueurs et annotations suivantes :

- La ligne qui commence par <<<<<<< marque le début de la zone conflictuelle.
- Le marqueur ======= représente la frontière entre les deux zones à fusionner.
- La ligne qui commence par >>>>>>> délimite la fin de la zone conflictuelle.

Exemple d'affichage d'un conflit :

```
<<<<<<< HEAD
library(tidyverse)
=======
library(tidyr)
>>>>>>> bugfix-tidyr
```

Note

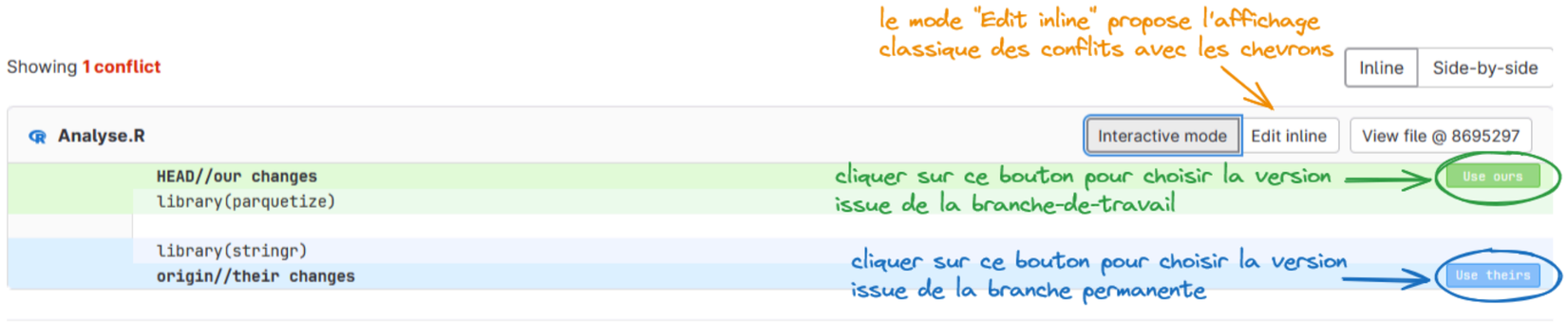
Pour identifier les conflits pendant un merge, la commande `git status` est très utile.

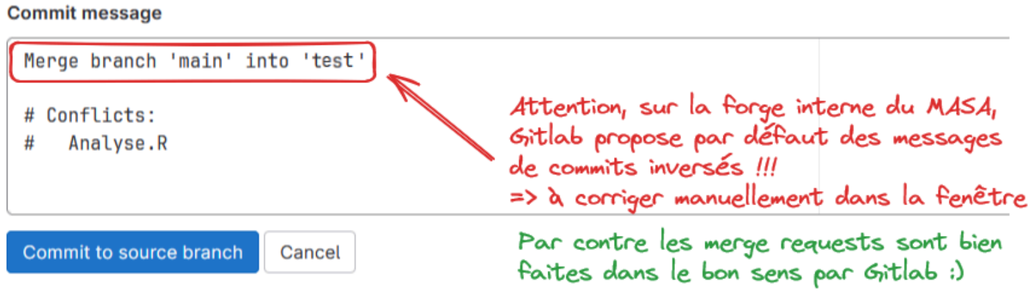
MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.28 Résoudre les conflits sur Gitlab

Pour les **"petits" conflits**, il est conseillé d'utiliser **Gitlab** (surtout pour les novices en Git).





MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.29 Résoudre les conflits localement avec le terminal

Pour résoudre **des conflits qui ont lieu sur plusieurs fichiers**, rapatrier les fichiers localement avec le terminal **pour les analyser sur RStudio** peut être une bonne idée.

Cependant les instructions proposées par Gitlab ne fonctionnent pas tout le temps.

Le DEMESIS vous propose de suivre la procédure suivante dans ce cas :

```
git checkout branche-de-travail
git pull origin main
git status
# Arbitrer dans le code :
# Supprimer les marqueurs de conflit à la main soit :
# Pour conserver la version distante, utiliser la commande : git checkout --theirs nom-du-fichier.
# Pour conserver la version locale, utiliser la commande : git checkout --ours nom-du-fichier.
git status
git add .
git commit -m "Résolution conflit"
git push origin banche-de-travail
```

=> Puis retourner sur Gitlab, rafraîchir la page de la merge request et le bouton de “merge” n’est plus bloqué ! 😃

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.30 Gérer les conflits (5/5)

Après une fusion, on peut **supprimer les branches temporaires** qu'elles soient distantes ou locales.

- Pour **les branches distantes**, elles sont supprimées dès lors que cette case est cochée dans Gitlab (sinon voir ici) :



- Pour supprimer **les branches locales** devenues inutiles, voir ici

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.31 Éviter au maximum les conflits

Quelques conseils :

- S'interroger sur la question de l'intersection des modifications et des fichiers potentiellement impactés lors de la planification des tâches
- Communiquer avec les autres membre de l'équipe pour dire qui travaille sur quoi
- La durée de vie des branches doit être la plus courte possible
- Modulariser son code (par ex. sous forme de fonctions)
- Résoudre les conflits le plus tôt possible et ne pas laisser la situation empirer.
- Faire des "bons" commits fréquemment

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.32 Exercice 5

Exercice 5 - Simulation d'un conflit

Reprendre l'exercice 4

- Dans votre projet R, créer 2 branches `bugfix-tidyverse` et `bugfix-tidyr`

- Se positionner sur la branche `bugfix-tidyverse` et corriger le script `Analyse.R` en modifiant l'unique ligne avec `library(tidyverse)`.
- Pousser la branche `bugfix-tidyverse` et la fusionner avec la branche permanente sur Gitlab (en supprimant la branche distante `bugfix-tidyverse`).
- Récupérer les résultats du merge dans le dépôt local sur votre branche permanente/principale.
- Supprimer la branche locale `bugfix-tidyverse` qui n'est plus utile
- Se positionner maintenant sur la branche `bugfix-tidyr` et corriger le script `Analyse.R` en modifiant l'unique ligne avec `library(tidyr)`.

- Pousser la branche `bugfix-tidyr` et la fusionner avec la branche permanente sur Gitlab.
- Normalement vous devriez faire face à un joli conflit 😄
- Résoudre le conflit

Source : exercice issue de cet article de blog

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.33 Exercice 6 collectif (1/2)

Exercice 6 (facultatif)

1. Se mettre par groupes de 2/3 personnes (qui appartiennet au même groupe sous Gitlab) : – une personne aura la responsabilité d'être mainteneur – une à deux personnes seront développeurs
2. Le mainteneur crée un dépôt sur Gitlab ; il/elle donne des droits aux développeurs du projet.
3. Chaque personne du groupe crée une copie locale (clone) du projet sous Cerise
4. Créer un fichier votre_nom-votre_prenom.md, écrire trois phrases de son choix, puis commit et pousse les modifications

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.34 Exercice 6 collectif (2/2)

Exercice 6 (facultatif)

A ce stade, une seule personne (la plus rapide) devrait ne pas avoir rencontré de rejet du push. C'est normal ! Avant d'accepter une modification, Git vérifie en premier lieu la cohérence de la branche avec le dépôt distant. Le premier ayant fait un push a modifié le dépôt commun ; les autres doivent intégrer ces modifications dans leur version locale (pull) avant d'avoir le droit de proposer un changement.

5. Pour ceux dont le push a été refusé, effectuer un pull des modifications distantes
6. Dans RStudio, afficher l'historique du projet et regarder la manière dont ont été intégrées les modifications des collaborateurs
7. Effectuer à nouveau un push de vos modifications locales
8. Les derniers membres du groupe devront refaire les étapes précédentes, potentiellement plusieurs fois, pour pouvoir push les modifications locales

Source : exercice issue de **ce support de formation** d'**@Inseefrlab**

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

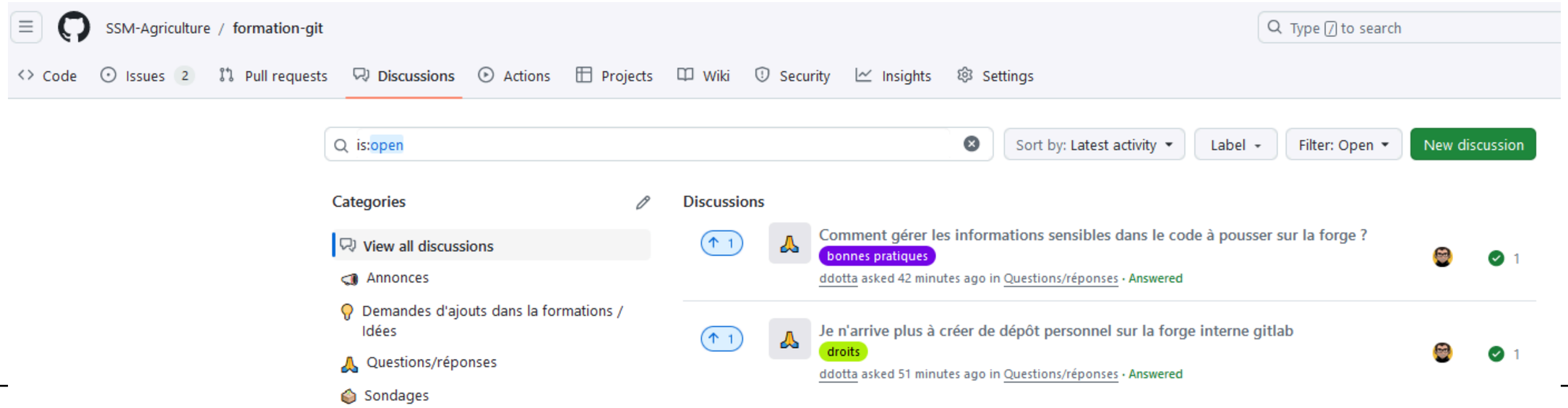
6 Pour en savoir plus

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

6.1 Ressources complémentaires ou pour chercher de l'aide

- Consultez **le wiki** de la formation qui contient :
  - Les corrections des exercices à télécharger et imprimer ;
  - Des ressources complémentaires (cheatsheet, tutoriels de connexion à Gitlab depuis Cerise…)
- Poser vos questions directement sur **l'espace discussions** de la formation.



MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

6.2 Bibliographie à consulter

En français :

- **Documentation officielle de Git en français**
- **Les fiches consacrées à Git dans utilitr**
- **Formation aux bonnes pratiques avec Git et R** par l'Insee
- **Tutoriel Git** par Atlassian

En anglais :

- **Happy Git and Github for the useR** par Jennifer Bryan (Posit)
- **Cheatsheet - Using Git and Github with RStudio**

MINISTÈRE
DE L'AGRICULTURE,
DE L'AGRO-ALIMENTAIRE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

6.3 Remerciements

Le DEMESIS remercie par avance tous les relecteurs/contributeurs à cette formation qui en feront un meilleur support pour la communauté française de R et en particulier pour les utilisateurs de RStudio.